Universidad Autónoma de Nuevo León

Facultad de Ingeniería Mecánica y Eléctrica

Interfaz de Monitoreo y Control de Condiciones ambientales de Fabrica de Plantas

Alumno: Julio de Jesús Moreno Sánchez / 1901410 / IMTC

Asesor: Dr. Efrain Alcorta García

A 20 de mayo 2024

Abstract (Aquí se describe, sintetiza y representa las principales ideas de un trabajo científico. Deberán ser aproximadamente entre 150 y 300 palabras.)

Uno de los desafíos que afronta la implementación de una fábrica de plantas yace en la medición y recopilación de datos; mismos con los que se puede analizar y gestionar el manejo de recursos o en general el desempeño del crecimiento de las plantas. Es en este contexto que se describe una propuesta de diseño y prototipado de un sistema embebido e interfaz digital inalámbrica para el monitoreo y descarga de lecturas ambientales como temperatura, humedad y concentración de dióxido de carbono, que además incorpora tecnologías de la industria 4.0 como IoT y servicios en la nube. Para desarrollarlo se requirió del uso de la placa de desarrollo ESP32 con el framework de Arduino, transductores para las lecturas y placas de prototipado en las que quedarían fijas las conexiones eléctricas. Por el lado del software se usaron librerías de código abierto para el microcontrolador que permiten integrar tanto los sensores como el envío de información a una base de datos. También se emplearon redes inalámbricas como wifi para la difusión de datos, y servicios en la nube de terceros como Google Firebase para el hospedaje de la página web y la base de datos.

La metodología utilizada fue *Design Thinking*, se definieron los requisitos de la toma de lecturas como tipo y cantidad de variables, medio de acceso, y en etapas posteriores se iban

desarrollando y añadiendo características escalonadamente a medida que iban funcionando. Para el desarrollo del código se utilizó GitHub ya que permite el control de versiones y la administración de cambios; mientras que, para el hardware tanto del contendor como del circuito, se hizo un CAD y sus respectivos diagramas eléctricos.
Palabras clave: Plantas, sensores, IoT, ESP32

Abstract: One of the challenges facing the implementation of a plant factory lies in the measurement and collection of data, which can be used to analyze and manage the management of resources or, in general, the performance of plant growth. It is in this context that we describe a proposal for the design and prototyping of an embedded system and wireless digital interface for monitoring and downloading environmental readings such as temperature, humidity, and carbon dioxide concentration, which also incorporates Industry 4.0 technologies such as IoT and cloud services. To develop it required the use of the ESP32 development board with the Arduino framework, transducers for readings and prototyping boards where the electrical connections would be fixed. On the software side, open-source libraries for the microcontroller were used to integrate both the sensors and the sending of information to a database. Wireless networks such as Wi-Fi were also used for data dissemination, and third-party cloud services such as Google Firebase were used to host the web page and the database.

Design Thinking was the methodology used followed by requirement definition for taking readings, such as type and number of variables, access media, and in later stages, features were developed and added as they started to work. For the development of the code, GitHub was used since it allows version control and change management; while, for the hardware of both the container and the circuit, a CAD and its respective electrical diagrams were made.
Key words: Plants, sensors, IoT, ESP32

1. *Introducción*

Las fábricas de platas del inglés (Plant Factory), también conocidas como granjas verticales son en sí una vertiente diferente en cuanto a la agricultura tradicional se refiere; de modo que se enfoca en la optimización del espacio y recursos de la producción agrícola, al permitir la creación sistemas de cultivos en interiores, verticalmente apilables y que además cuentan con un riguroso y optimizado control de las condiciones ambientales, necesarias para su crecimiento.

Además, este tipo de agricultura es perfectamente compatible con el paradigma de la Industria 4.0, en donde una gran cantidad de disciplinas eventualmente convergen para darle cabida. Disciplinas como informática industrial, comunicaciones, computación e inteligencia artificial, internet de las cosas (IoT), uso de sensores, big data, almacenamiento en la nube, entre otros; serían requeridos para garantizar el control operativo necesario de las condiciones ambientales que requerirán las plantas. Condiciones como iluminación, temperatura, suministro de nutrientes, humedad, pH, concentración de CO2, entre otros (Liu, y otros, 2021).

De las disciplinas anteriores las juegan un papel fundamental son el IoT "*que se refiere a en sí a la red de los objetos físicos, físicos como sensores, software y otras tecnologías con el propósito de intercambiar datos con otros sistemas y dispositivos utilizando el internet* "

(Oracle), siendo en este caso los dispositivos como principalmente microcontroladores o microprocesadores los capaces de realizar el intercambio de información, con sensores enfocados en las condiciones que requieren de ser monitoreadas u que fomentan el crecimiento de las plantas (sondas térmicas, sensores de gas humedad, entre otros).

Esta interconexión requiere de los llamados servicios en la nube, que no son otra cosa que "*la entrega de servicios de computación, como los servidores, almacenamiento, bases de datos, conectividad, software, análisis e inteligencia, sobre el internet (la nube), esto para proveer innovación rápida, recursos flexibles*" (Microsoft), viene a ser toda la infraestructura digital con la que se puede realizar el despliegue de aplicaciones, el medio por el cual la comunicación entre máquinas es posible.

Esta infraestructura no puede ser aprovechada por cualquier dispositivo, sino que se debe dispositivo que tenga la capacidad primeramente de conectarse a internet y en segundo la suficiente memoria y terminales para procesar información, realizar tareas y comunicarse con el mundo exterior.

Es de esta forma que se introduce lo que es el ESP32, que es un microcontrolador versátil desarrollado por Espressif Systems, destacado por su capacidad de conectividad inalámbrica, ya que integra tanto Wi-Fi como Bluetooth en un solo chip, lo que lo hace ideal para aplicaciones conectadas. También cuenta con un procesador de 32 bits LX6, que puede ser de uno o dos núcleos, con una frecuencia de reloj de hasta 240 MHz. Dispone de 520 KB de SRAM, 448 KB de ROM, y 16 KB de SRAM RTC, ofreciendo suficiente memoria para una variedad de aplicaciones (Teja, 2024).

Con esto en mente es que ahora se pueden discutir las siguientes secciones:

1.1. Planteamiento del problema

El problema que se plantea es el de poder diseñar una interfaz electrónica que permita la captura, modificación, envió y recepción de datos relacionados con variables ambientales relacionadas al crecimiento de plantas, empleando el microcontrolador ESP32 y en particular el módulo wifi que tiene incorporado, al igual que un medio que permita compartir y monitorear los datos obtenidos de dichas variables, a como lo puede ser una página web, teniendo en cuenta su programación.

1.2. Objetivo General:

Monitorear las variables de un rack humedad, temperatura, CO2, cantidad/calidad de luz, etc. vía remota usando las posibilidades WiFi del ESP32

1.3. Objetivos particulares:

- Diseñar una interfaz/sistema electrónico funcional que permita la recepción y envío de datos relacionados a variables ambientes de las plantas (humedad, temperatura, etc.) empleando el microcontrolador ESP32
- Programar una interfaz o medio que permita el monitoreo de las variables ambientales mediante wifi.
- Documentar hallazgos, planes, métodos, implementación y resultados, en un reporte formal.

2. *Metodología*

El proceso seguido para el desarrollo del dispositivo, parte de identificar que cada uno de los objetivos, agrega funcionalidades diferentes. Por lo que el desarrollo fue escalonado y avanzando de manera gradual. Difiriendo de lo que se estableció en un principio y ya que la interfaz y su contenido dependen del buen funcionamiento del hardware, este último fue lo primero en realizarse. Para ello fue necesario

recabar información de las distintas fuentes a modo de saber la estructura y capacidades tanto del microcontrolador usado como de los sensores y demás periféricos, con esto la lista de materiales también empezaba a definirse, y con el paso del tiempo se consolido en lo que es ahora; véase Tabla 1.

2.1. Sensores y Periféricos

El foco del proyecto gira entorno a las lecturas de los sensores, los sensores utilizados se seleccionaron debido a la compatibilidad en las señales y alimentación, siendo en su mayoría entre 3 y 5 V, con un bajo consumo de corriente; además son muy accesibles y baratos. Los sensores usados para la temperatura son los DS18B20, para humedad DHT11 y para concentración de CO2 QM135.

La primera fase involucró iniciar por poder tomar lecturas de los DS18B20, por lo que se emplea una protoboard para las pruebas, en la que se montan y rutean tanto el microcontrolador como los sensores; el código y circuito piloto (incluso para los sensores de humedad) fue extraído de uno de los sitios web con mayor contenido didáctico relacionado con proyectos de electrónica y en especial del uso del ESP32 con diferentes periféricos y tecnologías (Santos & Santos, randomnerdtutorials.com, 2019). Al igual también se consultó el repositorio de la librería del DS18B20 que incluye ejemplos y notas que guían acerca de cómo usarlo (Burton, y otros, 2020). Algo similar sucede con otra de las librerías que permiten que funcione el bus de datos "*OneWire*" y que tiene su sitio web de referencia (Studt, Pollard, James, & Stoffregen, 2017).

Una de las características más relevantes del uso del DS18B20 es que poseen un pin de salida digital, que se puede usar a modo de bus de datos, en conjunto con más sensores. Esto ahorraría el uso de terminales del micro. Con el código piloto listo, se obtuvo lo siguiente:

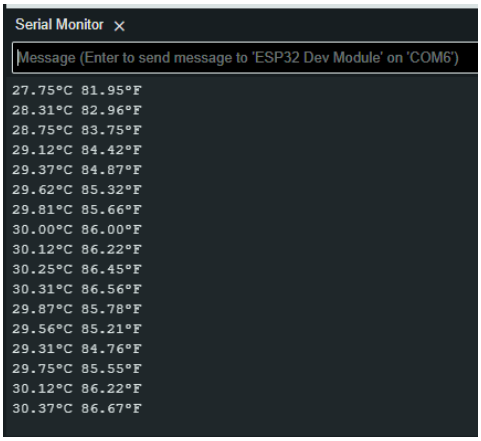


Figura 1: Código piloto, con un sensor usando Arduino IDE, y librería Dallas Temp (se muestra temperatura en distintas escalas).

```
Locating devices...Found 5 devices.
Found device 0 with address: 28721279A2000377
Found device 1 with address: 2851A296F0013C0B
Found device 2 with address: 28A52A79A200038C
Found device 3 with address: 28FF112B9315019C
Found device 4 with address: 28FF8B3B93150312
Temp1 C: 22.94  Temp2 C: 26.00  Temp3 C: 23.56  Temp4 C: 23.37  Temp5 C: 24.81
Temp1 C: 22.87  Temp2 C: 25.81  Temp3 C: 23.50  Temp4 C: 23.37  Temp5 C: 24.69
Temp1 C: 22.87  Temp2 C: 25.75  Temp3 C: 23.44  Temp4 C: 23.37  Temp5 C: 24.69
Temp1 C: 22.87  Temp2 C: 27.00  Temp3 C: 23.50  Temp4 C: 23.37  Temp5 C: 24.56
Temp1 C: 22.81  Temp2 C: 30.12  Temp3 C: 23.50  Temp4 C: 23.37  Temp5 C: 24.50
Temp1 C: 22.81  Temp2 C: 31.94  Temp3 C: 23.44  Temp4 C: 23.31  Temp5 C: 24.44
Temp1 C: 22.81  Temp2 C: 33.00  Temp3 C: 23.44  Temp4 C: 23.31  Temp5 C: 24.44
Temp1 C: 22.81  Temp2 C: 34.00  Temp3 C: 23.56  Temp4 C: 23.31  Temp5 C: 24.37
Temp1 C: 22.81  Temp2 C: 34.50  Temp3 C: 23.44  Temp4 C: 23.31  Temp5 C: 24.31
Temp1 C: 22.87  Temp2 C: 35.00  Temp3 C: 23.50  Temp4 C: 23.31  Temp5 C: 24.25
Temp1 C: 22.87  Temp2 C: 35.31  Temp3 C: 23.50  Temp4 C: 23.31  Temp5 C: 24.19
Temp1 C: 22.94  Temp2 C: 35.50  Temp3 C: 23.50  Temp4 C: 23.37  Temp5 C: 24.19
Temp1 C: 22.94  Temp2 C: 35.69  Temp3 C: 23.56  Temp4 C: 23.37  Temp5 C: 24.19
Temp1 C: 22.94  Temp2 C: 35.75  Temp3 C: 23.56  Temp4 C: 23.75  Temp5 C: 27.56
Temp1 C: 23.00  Temp2 C: 35.88  Temp3 C: 23.56  Temp4 C: 23.75  Temp5 C: 28.37
```

Figura 2: Código piloto, con 5 sensores usando Arduino IDE, y librería Dallas Temp.

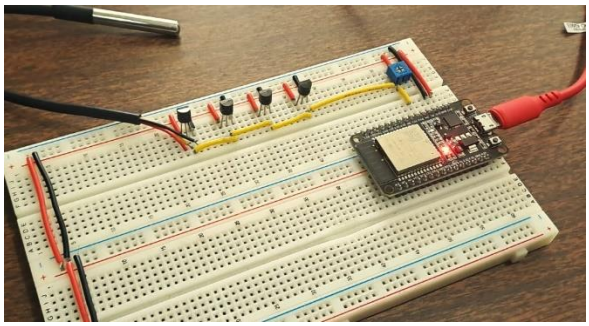


Figura 3: Montaje de micro y sensores

Tras lograr tomar lecturas de los sensores de temperatura, lo siguiente fue tomar las medidas del sensor DHT11; esto de manera aislada; el código de piloto también provino de sitio web de electrónica (Santos & Santos, randomnerdtutorials.com, 2019). Al articulo mencionado utiliza un sensor que también posee toma de temperatura, aun así, tenía variantes para usar el DHT22 (el recientemente mencionado) y

el que en realidad se terminó usando DHT11; la librería permite usar uno u otro. Usando solo los fragmentos pertenecientes a la humedad, se obtuvieron los siguientes resultados:

```
Humidity: 58.00
Humidity: 57.00
Humidity: 56.00
Humidity: 55.00
Humidity: 55.00
Humidity: 54.00
Humidity: 54.00
Humidity: 54.00
Humidity: 53.00
Humidity: 53.00
Humidity: 53.00
```

Figura 4: Lecturas de humedad DHT11 en Arduino IDE

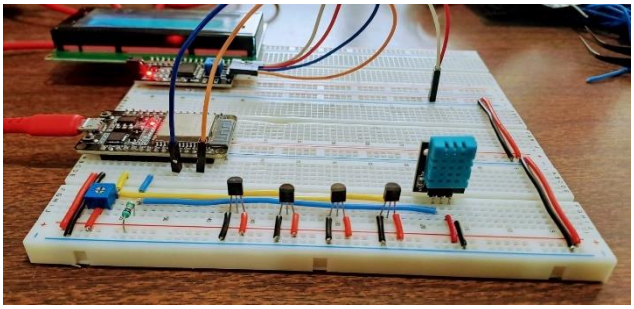


Figura 5: Montaje de DHT11 con ESP23, misma tabla de pruebas, pero con código aislado para solo medir humedad.

Para el sensor de concentración de CO2 QM135 el mismo proceso fe seguido, en este caso tanto código como diagramas de conexión adaptados al ESP32, estaban disponibles en el propio repositorio de la familia de sensores *QM* (Urquiza, y otros, 2022). La principal diferencia es en sí la conexión eléctrica, ya que estos sensores reciben 5 V y no los 3.3 que suministra el MCU. Los resultados de la prueba muestran todas las características evaluadas; el código final se limitaba a solo el valor de salida y no tanto en las configuraciones o calibraciones ya que no se contaban con ambientes controlados para realizarlas:

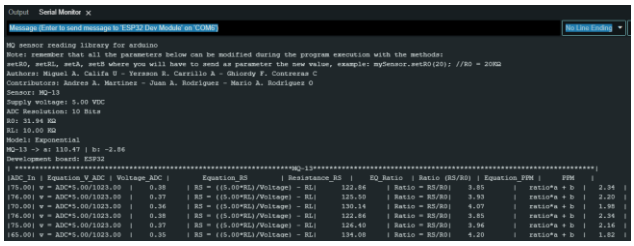


Figura 6: Prueba de código piloto extraído del repositorio oficial para lecturas de concentración de CO2.

Este mismo tipo de pruebas se realizaron con la pantalla LCD, para no depender del monitor serial, el código piloto proviene de dos sitios, ambos poseen una descripción del funcionamiento de la pantalla, fragmentos de código al igual que diagrama de conexiones (Santos & Santos, randomnerdtutorials.com, 2019), (lastminuteengineers.com).

Con las bases del funcionamiento establecidas para el uso de la pantalla LCD, se definió un layout que serviría para mostrar los valores y tipo de sensor en la propia pantalla, al principio colocando solo aquellos que cupiesen de manera predeterminada, mientras se actualizaban al realizarse nuevas lecturas, el layout utilizado se muestra en la Figura 7:

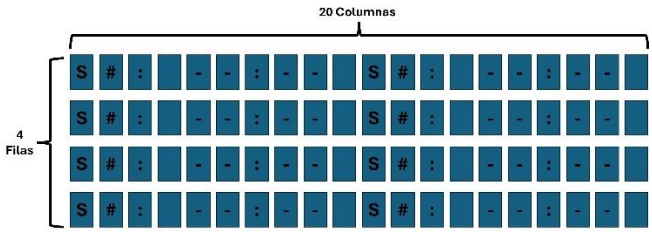


Figura 7: Distribución de signos en la pantalla LCD de 4 filas por 20 columnas.

En el layout la *S* viene a representar la letra característica o asociada a un tipo de sensor, de esta forma en la implementación se usa la *t* para temperatura, *h* para humedad y *g* para la concentración. El símbolo de numeral # se sustituye por el número de identificación de sensor, esto ya que, se más de un mismo tipo de sensor es utilizado; con esto se puede diferenciar uno de otro. Finalmente, el conjunto de caracteres "--:--" son en sí las celdas designadas para los valores de las lecturas de los sensores. En este

caso todos son números decimales de dos cifras, en dado caso que las lecturas no pudieran ser tomadas, el símbolo anterior las reemplazaría. Este mismo texto sería inducido por medio de un teclado matricial con el que se podrían dejar de mostrar lecturas de uno u otro tipo de sensor, al igual que usarlo para habilitar o deshabilitar el encendido de la pantalla.

Las pruebas pertinentes para ambos la LCD y el teclado son las siguientes:

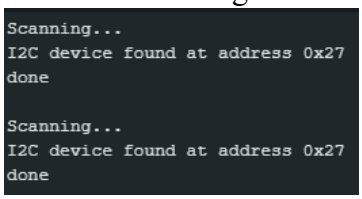


Figura 8: La comunicación con el ESP32 es por medio del protocolo I ^2C, es necesario usar un módulo I2C que permite utilizar dos pines y simplificar la transferencia de información; en este caso especificando el registro 0x27 y usando Arduino IDE.

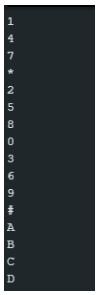


Figura 9: Con el teclado matricial se mostraron los respectivos valores mostrados en sus celdas, tras presionarlas, se imprimían por serial en el monitor de Arduino IDE.

Finalmente, la integración de los periféricos se muestra a continuación:

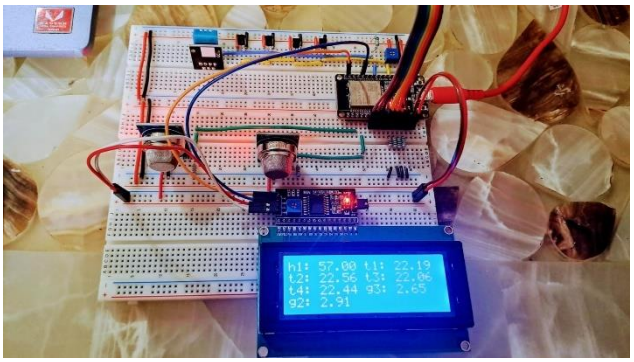


Figura 10: Integración de sensores con el ESP32 y la pantalla.

2.2. Ensamble de circuito

Una vez que la comunicación con los sensores está establecida y funciona apropiadamente lo siguiente por hacer es montar el circuito en una tablilla de conexiones y encapsularlo en un contenedor protector.

El traspaso del circuito desde la tabla de pruebas a la placa perforada requirió de adecuaciones ya que hay dos condiciones fundamentales que se deben cumplir: A) el circuito debe estar protegido por el contenedor y permitir tener sensores de manera externa, y B) los módulos y sensores usados ya cuentan con pines macho de conexión, así que debe adaptarse la conexión de módulos con pines.

Antes de pasar a la tablilla perforada, las propuestas de diagrama eléctrico fueron desarrolladas como esquemático y despues como circuito impreso en el entorno de diseño electrónico de la aplicación de Autodesk Fusion 360.

Para solucionar A) se optó por separar el circuito en dos placas, la primera contendría bloques de terminales a los que los sensores de podrían conectar, y de las conexiones saldrían terminales que se conectarían a la segunda placa que es la que contendría al microcontrolador y el resto de conexiones. Algo como lo siguiente:

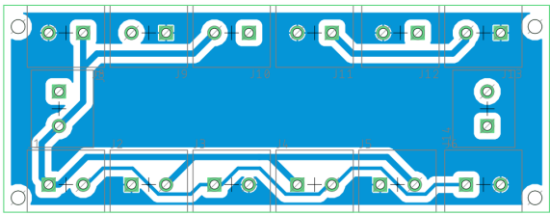


Figura 11: Placa con terminal blocks en toda la periferia para conectar sensores(escala no original).

El diagrama esquemático de conexiones se encuentra en el apartado 7.3.4

Y para la segunda placa, el principal factor que dificulta el armado sería la condición B), así que para solucionarla y en el proceso también A), la placa principal se le montaron pines tipo hembra a modo que tanto el ESP32, la pantalla LCD y el

resto de terminales de la placa de bloques, se conecten sobre estos; de esta forma las conexiones serían no solo más fáciles, sino que también limpias. El layout de la placa principal se vería como la Figura 12:

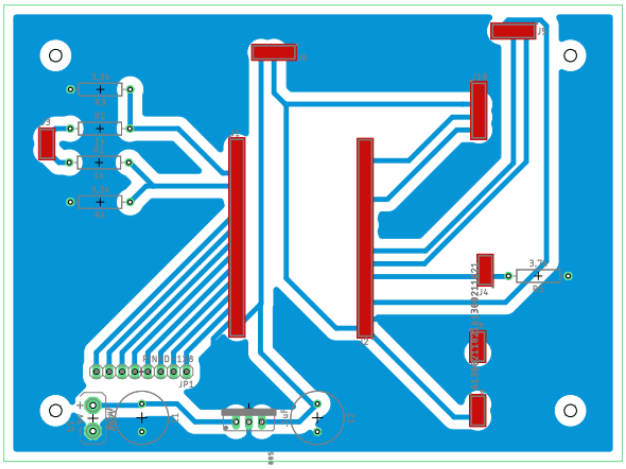


Figura 12: Layout de la placa principal (no a escala original)

(también véase el diagrama de conexiones en el apartado 7.3.4)

Si bien se aprecian más símbolos o elementos que en el protoboard, esto se debe a que, en el proto, la alimentación la suministraba el propio ESP32, y en el contendor se tendrán que usar pilas. El uso de pilas tuvo 2 vertientes, la primera usaba una de 3.7 V 2300 mAh, para energizar todo, el problema era que los MQ135 requieren 5V; entonces un cambio posterior se efectuó para que se use una pila de 9V 100 mAh; entonces se ocupó regular el voltaje con un regulador lineal 7805 y un par de capacitores, los cuales alimentan al ESP32 y sensores QM135, mientras que el ESP32 suministra 3.3 V al resto de sensores. Las placas ya soldadas se muestran a continuación:

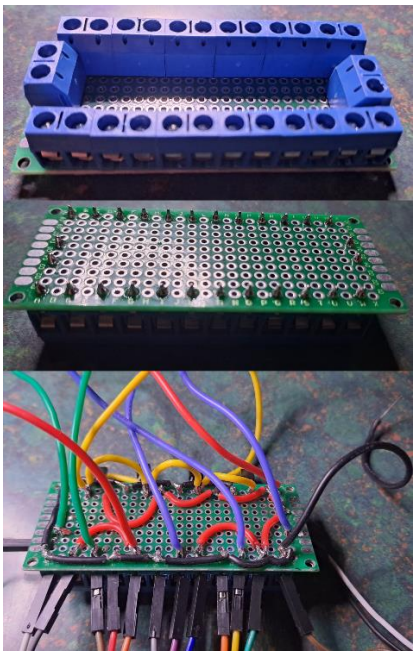


Figura 13: Placa Terminal block ya soldada

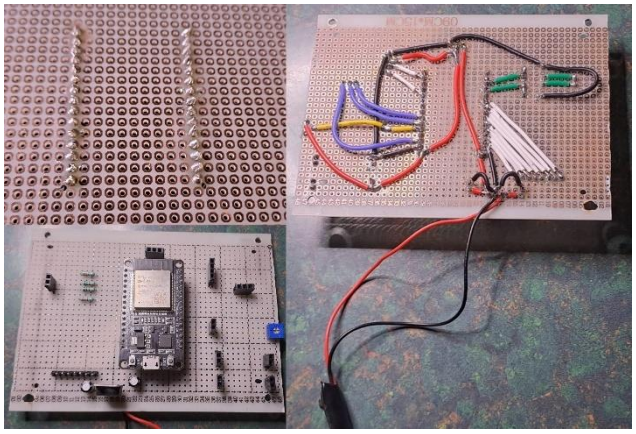


Figura 14: Placa principal adaptado para montaje de periféricos

2.3. Ensamble circuito-contenedor

Los criterios preliminares para efectuar el ensamble consideran que el contenedor posee un par de cavidades en las que se pueden empotrar elementos mediante tornillos, además se debe garantizar que las dimensiones del circuito coincidan con las del contenedor tanto para que quepa como para que pueda ser asegurado. El contenedor consta de dos tapas, que se sellan herméticamente con tornillos; este fue comprado y tiene la siguiente forma:

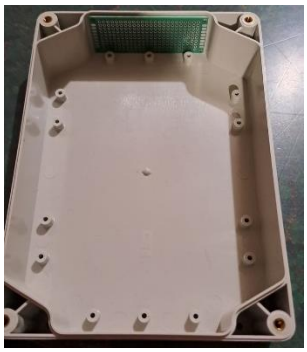


Figura 15: Tapa interior del encapsulado

Para determinar cómo ensamblarlos, se dimensionó el contendor con ayuda de un vernier y de las medidas, se hizo un modelo CAD para usarlo de referencia y explorar orientaciones y formas de ensamble del circuito y periféricos; Dibujos disponibles en el apartado 7.3.4. A continuación, se muestran vistas el CAD finalizado:

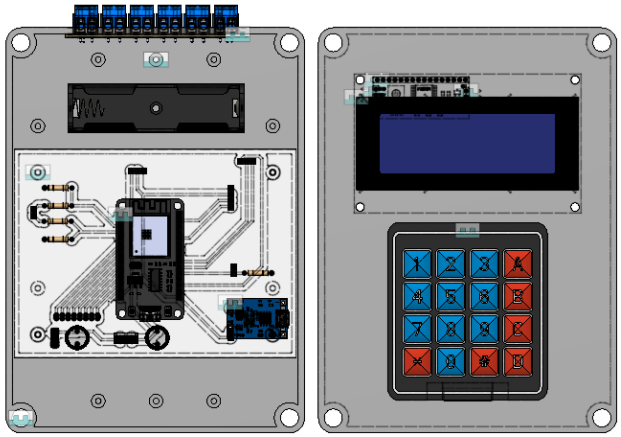


Figura 16: Vista superior del dispositivo

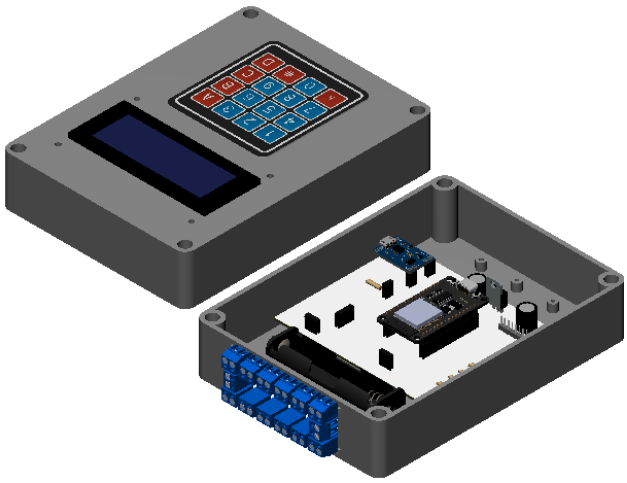


Figura 17: Vista isométrica (Conexiones no incluidas)

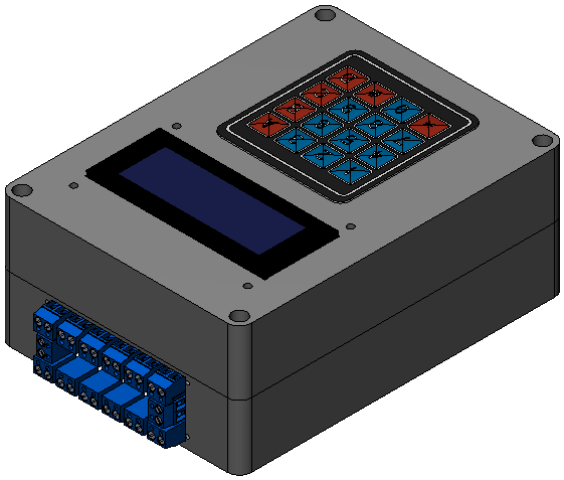


Figura 18: Vista isométrica (sin sensores y tapa empotrada)

De manera visual era evidente que la placa perforada de 15x9 cm se debía recortar de sus lado más largo para que quepa dentro del contender y que se pueda posteriormente perforar para la inserción de tornillos. Ademas para que al contendor se le puede poner tanto la pantalla LCD como el teclado y la placa de bloques, se tuvieron que hacer un par de perforaciones y cortes, los cortes fueron principalmente para empotrar la pantalla LCD, una ranura para que pasen las terminales del teclado matricial y otra ranura para que los cables de la placa de terminales puedan conectarse.

Las perforaciones fueron principalmente para atornillar la placa de terminales y la pantalla

LCD, también la placa con el circuito principal se perforo de modo que los agujeros coincidan y se pueda atornillar. Las dimensiones y posiciones de los cortes y agujeros también están especificados en los dibujos.

Con lo anterior el dispositivo se pudo ensamblar:

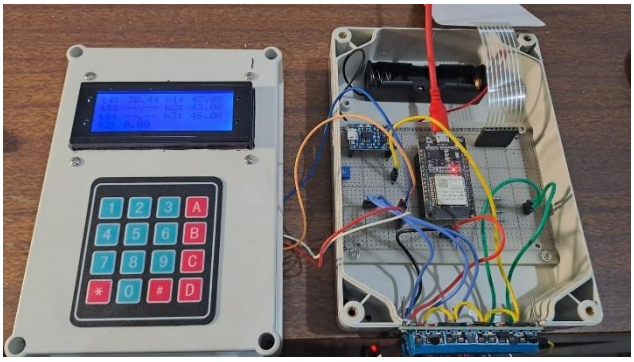


Figura 19: Dispositivo funcionando, armado y conectado

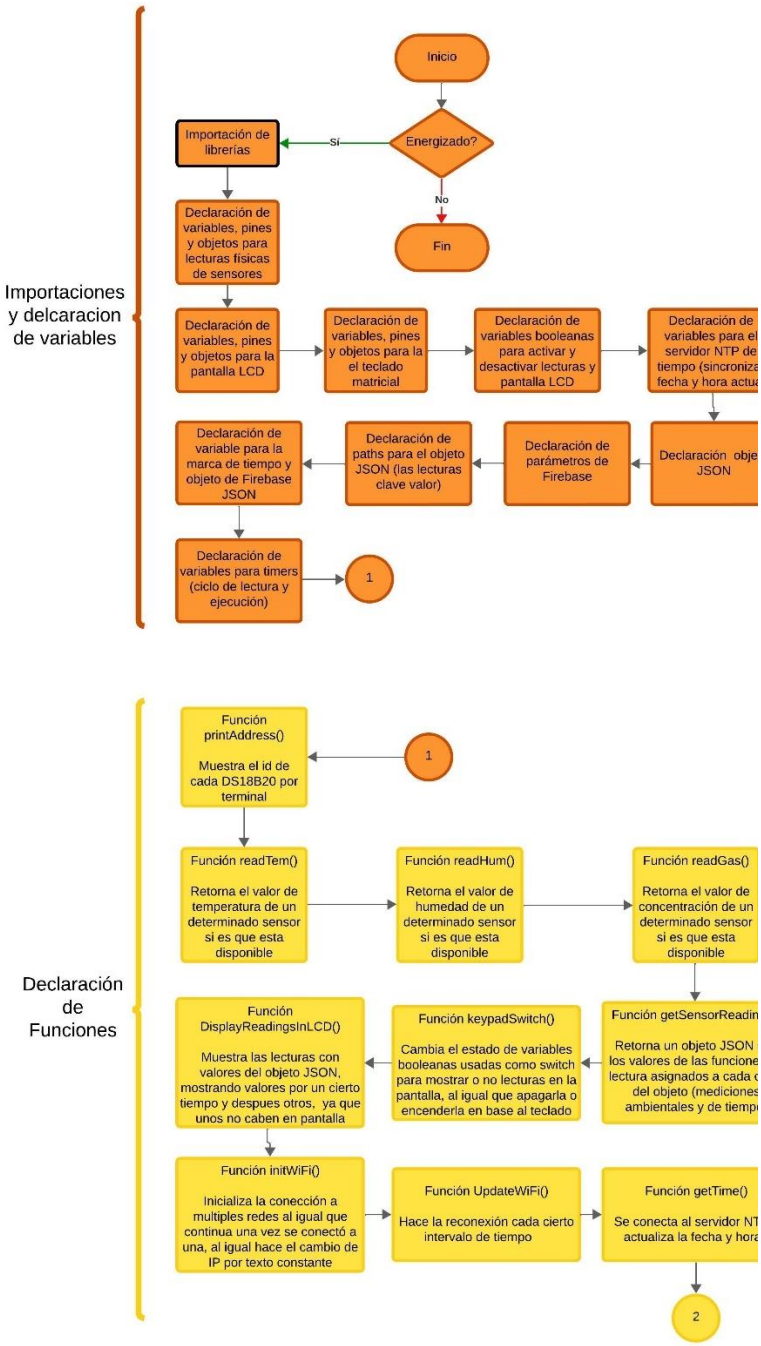
2.4. Interfaz digital y Wifi

Previo a adentrarse con la creación de páginas web y el uso de protocolos de comunicación inalámbrica, algunas de las capacidades del IDE de Arduino dificultaban el desarrollo; es por lo que se optó por migrar a otro editor de texto con mayor diversidad de herramientas para desarrollo de firmware para microcontroladores; dicho editor es *VS Code* en su versión para Windows. No se usó VS Code por sí mismo, sino también una extensión en específico: *PlatformIO,* que es descrita como "*una herramienta profesional multiplataforma, multiarquitectura y multimarco para ingenieros de sistemas embebidos y para desarrolladores de software que escriben aplicaciones para productos embebidos*" Según (Kravets, 2016) Doctor y CEO del software.

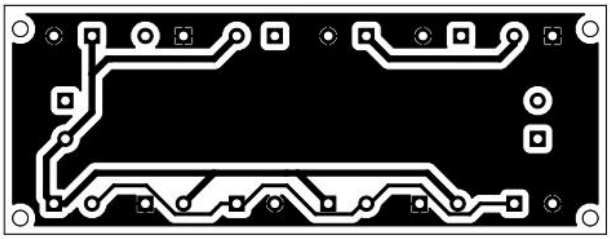
Es ampliamente usada para programar diversos tipos de microcontroladores de distintas familias como Arduino, ESP entre otros; con capacidad de programar en múltiples frameworks que para el caso del ESP32 son dos: Espressif o Arduino.

Cabe aclarar que el framework utilizado fue el de Arduino con PlatformIO, ya que las funciones, comandos y librerías empleadas tienen una mayor disponibilidad para ese entorno, e incluso es más accesible y sencilla la programación, comparado con ESP-IDE. Este último cuenta con su propia extensión en VS Code y se describe como "*el marco de desarrollo oficial para los SoC de las series ESP32, ESP32-S, ESP32-C, ESP32-H y ESP32-P.*" (Espressif), posee su propio sitio oficial con documentación y características al igual que repositorio en GitHub con contribuciones de más 800 personas. En la práctica las principales barreras presentadas fueron la curva de aprendizaje que supone programar con ESP-IDF; si bien el nivel de personalización de las características del micro y optimización de recursos es muy flexible, es necesario usar funciones de bajo nivel para alcanzar estos resultados (tanto con la extensión propia o al usarlo con PlatformIO), mismos que por el tipo de proyecto pueden ser alcanzados con el framework de Arduino; la lista final y completa de librerías usadas se puede encontrar en la

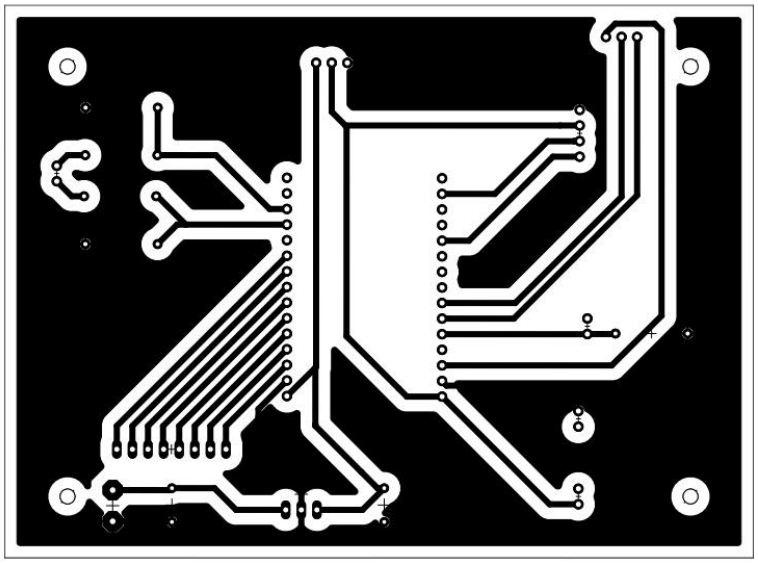
Diagrama_de Flujo – código ESP32



2.4.1. PCB Layout Terminal block



2.4.2. PCB Layout Circuito Principal



2.4.3. Diagramas de conexiones y Dibujos de ensamble

.

Otras ventajas de PlatformIO, indistintas del framework a usar, es que incluye monitor serial, opciones de verificación y carga del programa, se pueden usar tanto C como C++, para el caso de uso del MCU como host de un servidor web asíncrono, también la creación y carga de archivos mediante SPIFFS, definición

particiones de memoria de manera personalizada, carga de sketch de manera inalámbrica “OTA”, permite importación de librerías, uso de NodeJs para la creación de archivos de páginas web como *HTML*, *CSS* y *JavaScript* al igual que vinculación con servicios web, compatibilidad con Git y GitHub para la administración de versiones y trabajo del proyecto para cargar sus cambios a un repositorio.

Debido a estas características y el hecho de que dentro de los atributos del ESP32 este la conexión Wi-Fi, fue sencillo identificar una librería capaz de permitir la creación de una interfaz, en este caso mediante una página web.

El concepto de la interfaz como página web pretende ser sencilla de acceder y leer, siendo solo un apoyo visual de la toma de lecturas; en cuanto a logística interna cambio varias veces, mientras que la forma apariencia tabular se mantenía, a continuación, se muestra un boceto inicial de la página web que más tarde pasaría a materializarse mediante HTML:

Configuraciones		
Modulo		ESP32
Pin #	Descripción	Tipo
GPIO #	$#&"#$#"$5ge	Entrada/salida

Variables ambientales					
Temperatura		Humedad		CO2	
Sensor #	Lectura	Sensor #	Lectura	Sensor #	Lectura
1	--:--	1	--:--	1	--:--
2	--:--	2	--:--	2	--:--
3	--:--	3	--:--	3	--:--

Descargar

Figura 20: Boceto de la interfaz de monitoreo

La estructura general del archivo html tiene incluye dentro de sí las etiquetas convencionales al igual que una serie de contenedores como el propio cuerpo de la página, divisores para delimitar las secciones, dentro de estos se tiene un encabezado que sugiere de que trata la tabla, y en el mismo escalón, la propia tabla con sus filas, algunas definidas como encabezados y otras como celdas con contenido relacionado al encabezado. También se añadieron los enlaces al archivo de estilos CSS y una sección de enlaces para los archivos de JavaScript. El primero viene a ser la apariencia de la página web y el segundo es el comportamiento o todo aquello denominado como “*backend*”.

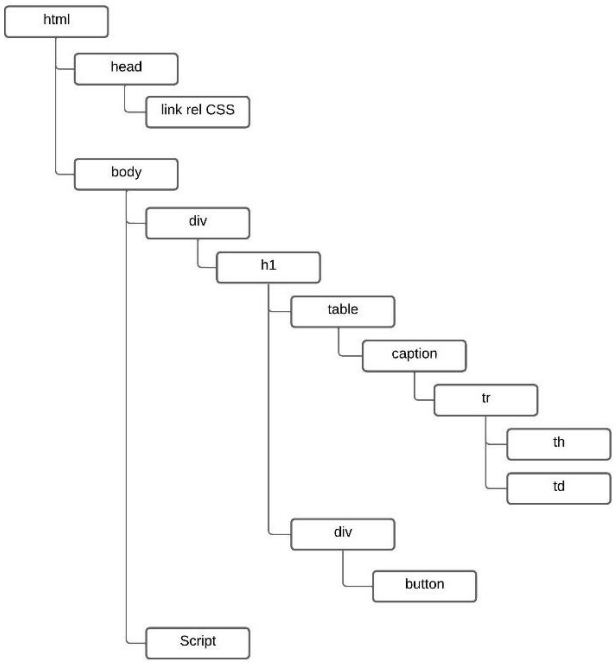


Figura 21: Jerarquía de elementos en HTML

Para el diseño de la página se utilizó como recurso una serie de videos que detallan conceptos tanto de html como de css (Palomares, 2021), esto de manera introductoria. Cabe mencionar que en la Figura 21 aparece un botón, este esta pensado que realice la descarga de datos, pero en las etapas iniciales solo forma parte del boceto, y en las etapas más tempranas de la página web, no estaban incorporados todavía.

Aspecto de la página creada desde Visual Studio Code:

Environmental Monitor

Settings Table

Board:	ESP32 Dev Module	
Pin number	**Pin Description**	**Pin Type**
GPIO 4	Temperature reading bus	input
GPIO 5	Humidity readings	input
GPIO 34	CO2 readings sensor$_{1/2}$	input
GPIO 39	CO2 readings sensor$_{2/2}$	input

Sensors' Readings

Temperature		Humidity		CO2	
Sensor ID	**Reading (°C)**	**Sensor ID**	**Reading (%)**	**Sensor ID**	**Reading (ppm)**
1	t1	1	h1	1	g1
2	t2				
3	t3	2	h2		
4	t4			2	g2
5	t5	3	h3		
6	t6				

General Info

Figura 22: Prototipo de monitor de variables ambientales con html, css y js.

2.5. Primer alternativa de interfaz

Vincular la página web con el ESP32 llevó a la exploración de alternativas similares en resultados, pero diferentes a la implementación. La primera opción a probar fue implementar la estrategia *AJAX*, misma que es descrita como ” *(Asynchronous JavaScript and XML o JavaScript Asíncrono y XML) es un conjunto de técnicas usadas para realizar cambios asíncronos en una página web. Esto nos permite actualizar objetos de un sitio que pueden ser texto o números, por ejemplo, sin tener que recargar la página*” y que además complementa con código de ejemplo el artículo (Mera, 2022). Es decir, que se pueden crear páginas web manera asíncrona usando el ESP32, requiriendo librerías como *ESPAsyncWebServer*, *AsyncTCP*, y *wifi*; de esta forma el ESP32 se convierte en el host o bien servidor de la página web, mientras que el cliente puede hacer solicitudes al servidor usando el protocolo HTTP desde un navegador y si está conectado a la IP que genera el ESP32.

La página web cómo tal esta embebida dentro del código del MCU a modo de arreglo de caracteres, de esta forma el micro lee y carga cada una de las líneas de la página y emite una IP que al colocarla en el navegador redirige a dicha página (Esto de modo simplificado). Al estar embebido ya sea como un archivo importado o dentro del archivo principal, la redacción de la página requiere que tanto los estilos CSS, la estructura HTML y las funciones de JavaScript, estén también dentro del cuerpo de la página. El siguiente diagrama de la Figura 23 describe lo anterior:

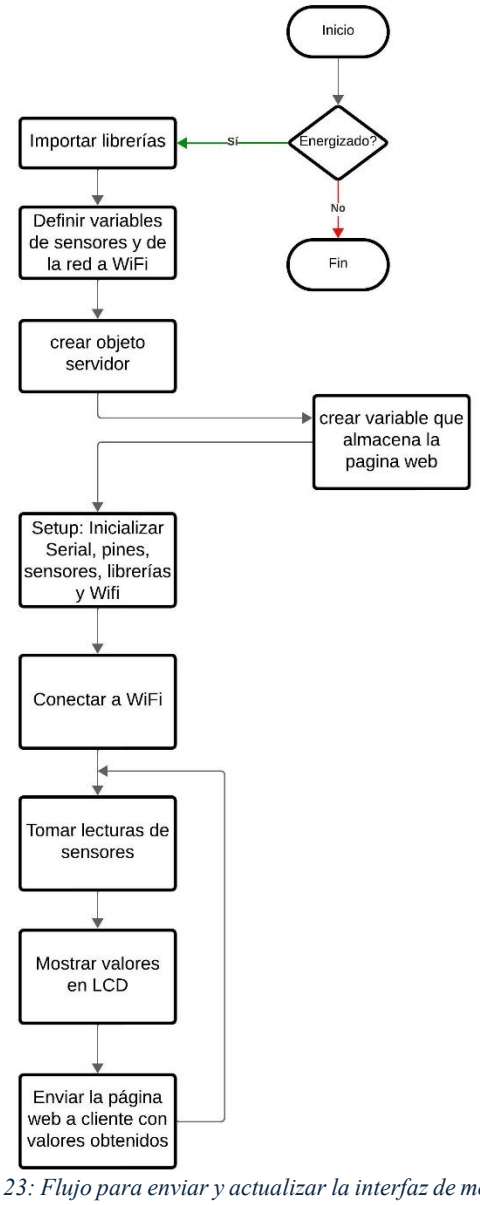


Figura 23: Flujo para enviar y actualizar la interfaz de monitoreo con AJAX

Aquí el envío de la página también implica que los valores de ciertos textos dentro de la misma

se sustituían por las lecturas tomadas. Esto ya permitía una interacción como la siguiente:

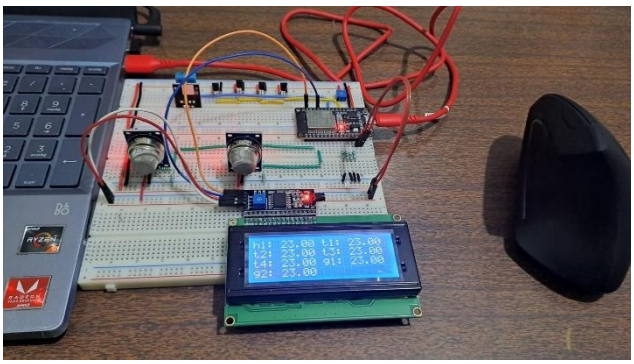


Figura 24: Prueba de hardware y monitor utilizando valores de prueba

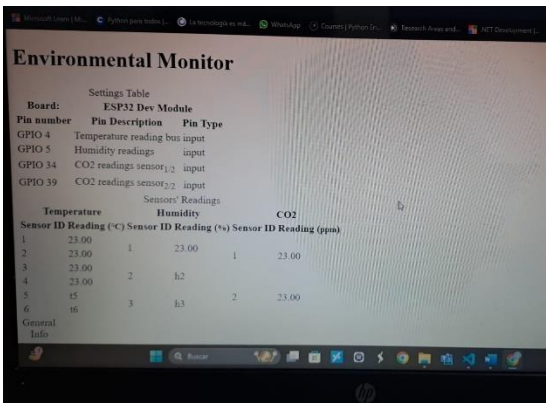


Figura 25: Monitor mostrando los valores de prueba

2.6. Segunda opción de Interfaz

La segunda opción no afecta el funcionamiento de manera sustancial, la principal diferencia es que ahora se utiliza el sistema de archivos SPIFFS, en el cual se puede cargar archivos de diferente extensión, de esta forma la página web se puede programar de manera tradicional y sin estar embebida en el sketch del ESP32. Al igual con el sistema de archivos se puede acceder al contenido de los mismos y en este caso seguir enviando la página al cliente junto con la actualización de valores; para lograrlo se recurrió nuevamente a los autores mayormente mencionados hasta ahora, visitando los artículos que hablan tanto de la carga de archivos con SPIFFS (Santos & Santos, 2021) y la instalación del sistema de archivos (Santos & Santos, randomnerdtutorials.com, 2018).

Otro cambio importante fue el uso de la librería *Arduino_JSON* la cuál permitía crear un objeto de este tipo y almacenar a través de paths o direcciones tipo árbol, los valores a modo de clave valor; la documentación de su uso esta también en su repositorio correspondiente (Mistry, y otros, 2022)

También se añadió una característica con la que se puede conectar a múltiples redes, y en lugar de esperar que el MCU comparta una IP, que esta quede definida como un texto (similar a un dominio, pero local) que sustituya la IP y facilita la conexión en el navegador web. Para este punto se enfatiza que el guardado de lecturas todavía no era posible, y se exploraba la idea de utilizar una memoria SD para hacer el guardado local de datos. El algoritmo en este punto se veía algo así:

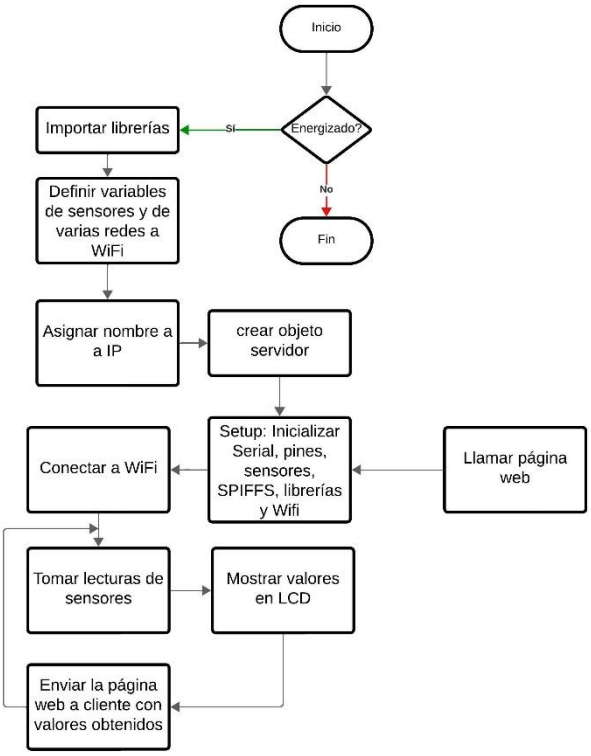


Figura 26: Algoritmo incluyendo conexión a múltiples redes, cambio de IP por nombre constante y uso de SPIFFS

Esta versión estable es de las primeras en cargarse al repositorio formal del prototipo, ya que los cambios eran cada vez mayores, mismo que es

accesible en **Error! Reference source not found. Error! Reference source not found.**.

2.7. Tercera opción de interfaz

Finalmente, el la última versión del código, hay algunas funciones que dejan de ser necesarias al sustituirlas por los servicios de *cloud computing* de Google a como lo es Firebase.

Firebase cuenta con planes de suscripción, el más básico es gratuito y compatible con los requisitos y alcance de este proyecto, algunos de ellos son creación de bases de datos de tiempo real, con capacidad de 1 GB, 100 conexiones simultaneas (clientes) y descarga de 10 G al mes; permite hospedar páginas web de 1G de espacio, 5 G de almacenamiento, otro tipo de base de datos llamada Firestore, la cual permite guardar no solo valores sino archivos en general (1 G de espacio), capacidad de autentificación con un límite de 10000 verificaciones por celular, e ilimitado para otros medios según se muestra en el sitio oficial (Google).

Las herramientas utilizadas de Firebase fueron 3 principalmente: RTDB (real time database), Hosting y Authentication. Para poder hacer esta vinculación, de primeras se requiere la librería para ESP32 de Firebase, que permite crear un objeto, en este caso de tipo JSON, el cual utiliza corchetes para indicar la jerarquía tipo árbol que existe entre los objetos que la integran. Este tipo de estructura es la que permite Firebase para organizar la información, cada uno de los elementos puede ser un nodo dentro de esa estructura, y para acceder a ellos se utilizan direcciones, de forma similar a como con los directorios en el PC. La parte de la autentificación es una capa de seguridad ante la escritura y lectura de datos desde y hacia la base de datos, esta se configura desde la propia pagina web de Firebase, mientras que la estructura de la base de datos se define en este caso desde el código.

La jerarquía que se definió es que, por cada usuario, se tenga una colección de lecturas, mismas que se identifican por una marca de tiempo, esta marca actuaría como índice único e irrepetible que a su vez contendría las lecturas realizadas en cierto instante de tiempo; la estructura es la siguiente:

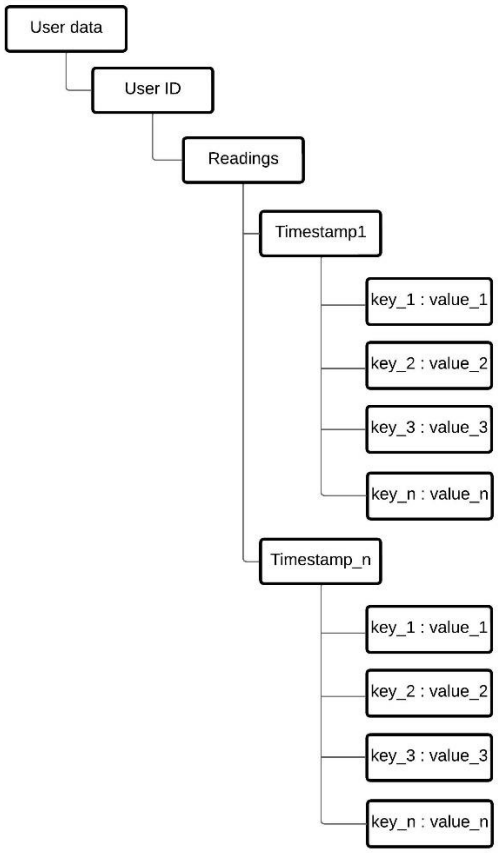


Figura 27: estructura del objeto JSON RTDB. Timestamp se refiere a la marca de tiempo, por cada marca de tiempo se tienen anidados

Como se observa en la Figura 27 la marca de tiempo "*Timestamp*" tiene anidados las lecturas pertenecientes a dicho instante de tiempo.

Las funciones que pasan a no usarse son por un el sistema de archivos SPIFFS, ya que no es necesario que el ESP32 genere el servidor, para eso ya se cuenta con el hosting de Firebase. Por lo mismo tampoco se requiere usar el servidor asíncrono en el ESP32 "*ESPAsyncWebServer*", lo que consume incluso menos recursos del MCU. Diagrama de flujo del ESP32 en el apartado 7.3.1.

Para incorporar Firebase al microcontrolador a parte de la librería, también fue necesario utilizar un correo de Google para utilizar los servicios, y desde el modo desarrollador, crear un proyecto (Santos & Santos, randomnerdtutorials.com, 2022)

La librería de Firebase funciona solo con el ESP32, para incorporar Firebase con la página web e incluso para que esta quede hospedada, lo que se requiere es por un lado utilizar Node.js que "*es un entorno de ejecución de JavaScript multiplataforma, de código abierto y gratuito que permite a los desarrolladores crear servidores, aplicaciones web, herramientas de línea de comandos y scripts*" (nodejs). Esta herramienta se instala en el equipo y despues se descargan los recursos relacionados a Firebase, con esto se da forma a la aplicación web; la base nuevamente tomada como referencia del sitio web (Santos & Santos, randomnerdtutorials.com, 2021).

Cabe recalcar que la estructura de la página web mediante html es indispensable y preferible que se realice antes de conectar con Firebase, aunque no forzoso.

De manera simplificada el flujo de información es el siguiente:

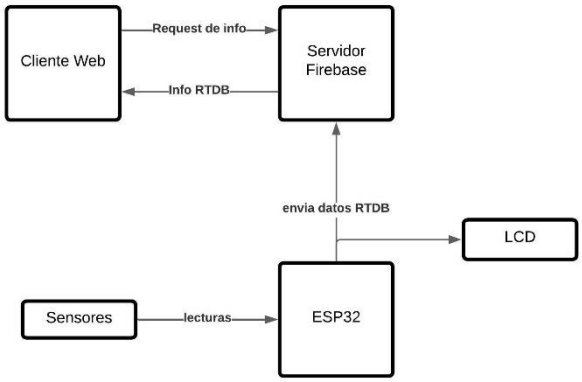


Figura 28: Flujo de información desde sensores hasta Interfaz

En cuanto al algoritmo que describe como es el intercambio de información entre el cliente Web y el servidor se presenta el diagrama de la

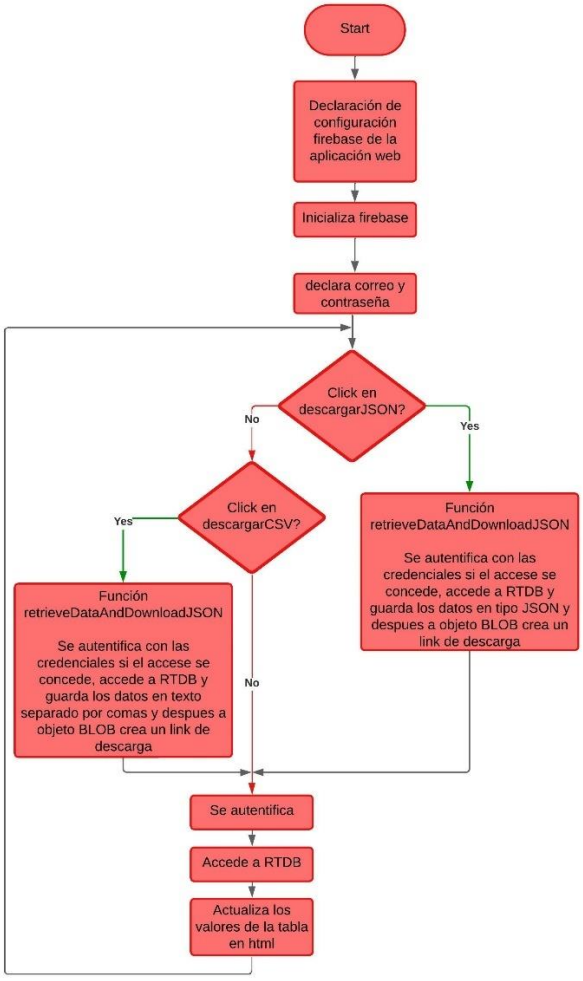


Figura 29: Solicitudes ida y vuelta incorporadas con JavaScript y Firebase para la interacción de la página web

Como se menciona en el diagrama, en las pruebas anteriores no había modo de descargar información, ahora ya se añadieron botones para que toda la información se descargue, ya sea en formato JSON o en formato CSV. Los últimos cambios que se le hicieron a la página web además de los botones fueron de tipo cosmético, adaptando los colores y tamaños a la paleta de colores del usuario final desde el archivo CSS:

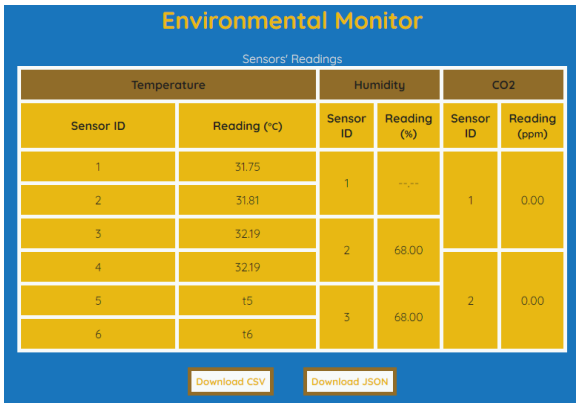


Figura 30: Aspecto final de la aplicación Web

3. *Discusión de resultados*

Comparando el todo el proceso de desarrollo contra los objetivos propuestos, se puede afirmar que el desarrollo del prototipo fue exitoso, aunque no perfecto, hay muchas áreas de oportunidad para mejorar, una de ellas es el propio encendido del dispositivo, ya que no se contempló una vez todo el circuito estuviera cubierto por el contenedor; del mismo modo los sensores están limitados, en el sentido de que no se pueden adaptar más de los que se permitió debido al propio hardware.

En cuanto al encapsulado, este al ser genérico requiere de realizarle cortes y perforaciones, lo que toma tiempo y herramienta especializada, de lo contrario es probable dañarlo e inutilizarlo.

En cuanto a la aplicación web, esta no tiene ningún menú para navegar o funciones extra que puedan ser de utilidad como registros etc.

Incluso la pantalla que muestra las lecturas deja algo que desear, ya que la interacción del teclado matricial controla muy pocas características y estas debido al modo de programación requieren algo de tiempo en ejecutarse. Aun así, también son necesarias más pruebas para terminar de detectar los errores o posibles fallos a corregir.

Independientemente de lo anterior, el proyecto como un todo se pudo realizar y principalmente ser funcional, ejemplo de ello es que a continuación se muestran los resultados de la información capturada en la base de datos al igual tras la descarga y vista desde Visual Studio Code:

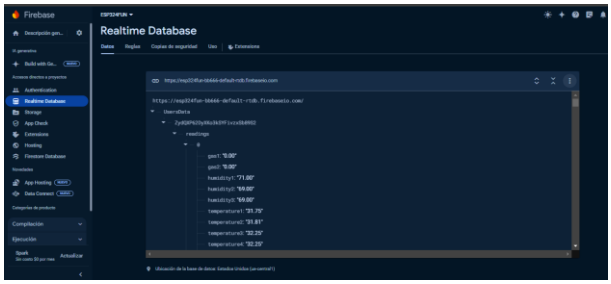


Figura 31: Datos RTDB

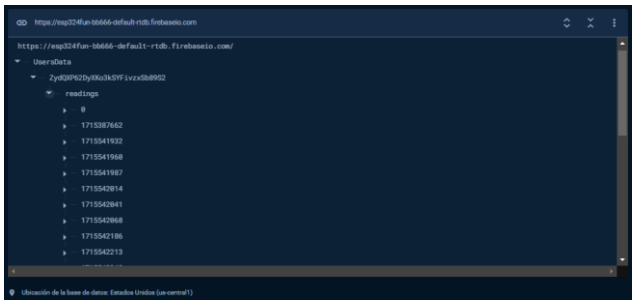


Figura 32: Timestamps ocultos para mayor visibilidad

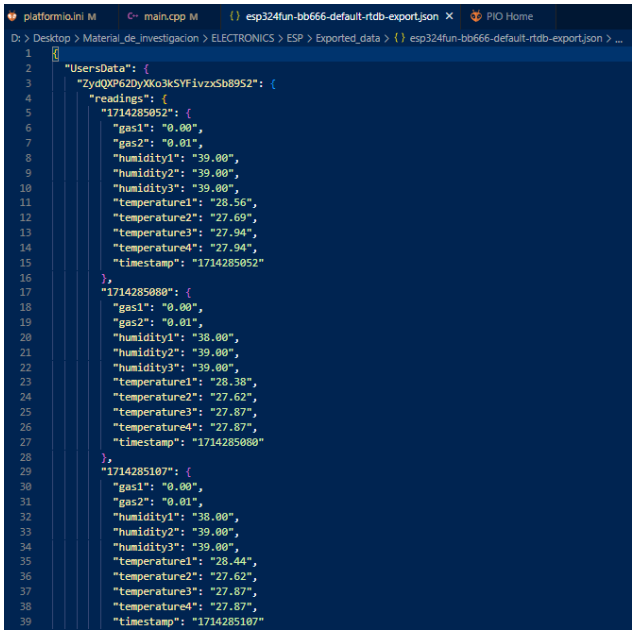


Figura 33: Archivo JSON descargado de RTDB web app

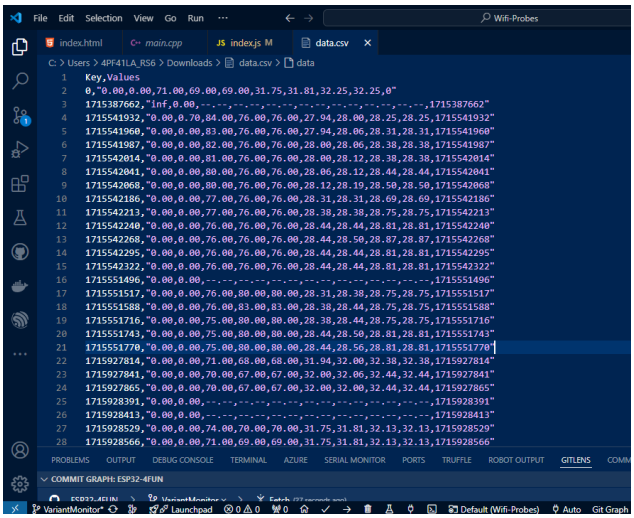


Figura 34: Archivo CSV descargado de RTDB web app



Figura 35: Dispositivo recién energizado

Algunas de las mejoras serían que, en lugar de utilizar un contendor genérico, que se haga un impreso para mayor precisión y facilidad de ensamble. Y para las conexiones eléctricas, sería hacer un circuito impreso en toda regla, para disminuir la cantidad de cables.

4. *Conclusiones*

De primeras una de las cuestiones más importantes al momento de real realizar lo que es la integración tanto de la aplicación web como del hardware es tener una vista previa o conjuntos de herramientas compatibles entre sí. Esto permite una mayor flexibilidad ante posibles retos durante dicha integración al igual que sirve para comparar cuál sería la ruta más rápida o beneficiosa durante todo el desarrollo. Ejemplo de ello es la cantidad de bibliotecas de desarrollo para microcontroladores, e incluso los propios microcontroladores que cada vez están dotados de mayor versatilidad en cuanto a componentes, sensores y módulos con los que interaccionan.

Actualmente existen demasiadas tecnologías y herramientas por ejemplo para lo que son las aplicaciones web, por lo que ésta es sólo una forma de hacerlo y en realidad pudiera haber un montón de variantes cada una con sus propios retos ventajas y desventajas. Desde el punto de vista de los sensores actualmente existen muchas formas de medir magnitudes físicas entonces es cada vez más demandante la disposición de tecnologías para el manejo de sus datos y que como en esta ocasión se implementó particularmente para censar magnitudes ambientales no hay límites en cuanto a los contextos en los que se puede aplicar.

Al igual esto supone que la creación de estas tecnologías es cada vez más accesible y asequible. Desde el momento e que aprender y disponer de material de dominio publico como el código libre.

Relacionado a la metodología, es claro que el haber podido definir la estrategia a seguir, previo a su ejecución, permitió concretar el prototipo.

5. *Contribución del estudiante*

- Investigación preliminar de fábricas de plantas, tecnologías involucradas y tendencias.

- Asistencia con el diseño y selección de la interfaz electrónica (microcontroladores, teclados matriciales, pantalla LCD, entre otros).
- Asistencia en el diseño de la lógica de control (parámetros etc.)
- Programación del microcontrolador, (pines, aplicación de la lógica de control, envió y recepción de datos con el módulo wifi).
- Montaje de la Interfaz.
- Monitoreo y control remoto de las variables ambientales (mostrar los datos en una página web).
- Documentación.

6. *Referencias bibliográficas*

Burton, M., Tillaart, R., gitkomodo, G., K., Kravets, I., Stehlík, P., . . . Shatz, H. (Septiembre de 2020). *github.com.* Recuperado el 18 de Mayo de 2024, de Arduino-Temperature-Control-Library (3.9.1): https://github.com/milesburton/Arduino-Temperature-Control-Library

Espressif. (s.f.). *docs.espressif.com.* Recuperado el 18 de Mayo de 2024, de ESP-IDF Programming Guideℑ: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/index.html

Google. (s.f.). *firebase.google.com.* Recuperado el 18 de Mayo de 2024, de Pricing plans: https://firebase.google.com/pricing

Kravets, I. (5 de Diciembre de 2016). *docs.platformio.org.* Recuperado el 18 de Mayo de 2024, de What is PlatformIO?: https://docs.platformio.org/en/latest/what-is-platformio.html

lastminuteengineers.com. (s.f.). Recuperado el 18 de Mayo de 2024, de Interface an I2C LCD with Arduino: https://lastminuteengineers.com/i2c-lcd-arduino-tutorial/

Liu, Y., Mousavi, S., Pang, Z., Ni, Z., Karlsson, M., & Gong, S. (27 de deciembre de 2021). *Plant Factory: A New Playground of Industrial Communication and Computing.* Recuperado el 29 de noviembre de 2023, de www.mdpi.com: https://www.mdpi.com/1424-8220/22/1/147

Mera, H. (26 de Abril de 2022). *todomaker.com.* Recuperado el 18 de Mayo de 2024, de Cómo usar AJAX en un ESP32: https://todomaker.com/blog/como-usar-ajax-en-un-esp32/

Mistry, S., per1234, Entinger, A., van Gulik, D.-W., Mueller, M., & Dixit, M. (22 de Noviembre de 2022). *github.com.* Recuperado el 18 de Mayo de 2024, de Arduino_JSON: https://github.com/arduino-libraries/Arduino_JSON

nodejs. (s.f.). *nodejs.org.* Recuperado el 18 de Mayo de 2024, de Run JavaScript Everywhere: https://nodejs.org/en

Palomares, K. (21 de Mayo de 2021). *www.youtube.com.* Recuperado el 18 de Mayo de 2024, de El Mejor CURSO GRATIS de HTML desde Cero [Completo y Actualizado]: https://www.youtube.com/watch?v=CGV9CuIUtwk&list=PLnunbwZjHqMOFL0vGRg-2OhV8fTtbujrD

Santos, R., & Santos, S. (2019). *randomnerdtutorials.com.* Recuperado el 18 de mayo de 2024, de ESP32 DS18B20 Temperature Sensor with Arduino IDE (Single, Multiple, Web Server):

https://randomnerdtutorials.com/esp32-ds18b20-temperature-arduino-ide/

Santos, R., & Santos, S. (2019). *randomnerdtutorials.com.* Recuperado el 18 de Mayo de 2024, de ESP32 with DHT11/DHT22 Temperature and Humidity Sensor using Arduino IDE: https://randomnerdtutorials.com/esp32-dht11-dht22-temperature-humidity-sensor-arduino-ide/

Santos, R., & Santos, S. (2019). *randomnerdtutorials.com.* Recuperado el 18 de Mayo de 2024, de How to Use I2C LCD with ESP32 on Arduino IDE (ESP8266 compatible): https://randomnerdtutorials.com/esp32-esp8266-i2c-lcd-arduino-ide/

Santos, R., & Santos, S. (2021). *randomnerdtutorials.com.* Recuperado el 18 de Mayo de 2024, de ESP32 with Firebase – Creating a Web App: https://randomnerdtutorials.com/esp32-firebase-web-app/

Santos, R., & Santos, S. (2022). *randomnerdtutorials.com.* Recuperado el 18 de Mayo de 2024, de ESP32 Data Logging to Firebase Realtime Database: https://randomnerdtutorials.com/esp32-data-logging-firebase-realtime-database/

Santos, S., & Santos, R. (2018). *randomnerdtutorials.com.* Recuperado el 18 de Mayo de 2024, de Install ESP32 Filesystem Uploader in Arduino IDE: https://randomnerdtutorials.com/install-esp32-filesystem-uploader-arduino-ide/

Santos, S., & Santos, R. (2021). *randomnerdtutorials.com.* Recuperado el 18 de Mayo de 2024, de ESP32 with VS Code and PlatformIO: Upload Files to Filesystem (SPIFFS): https://randomnerdtutorials.com/esp32-vs-code-platformio-spiffs/

Studt, J., Pollard, T., James, R., & Stoffregen, P. (14 de April de 2017). *www.pjrc.com.* Recuperado el 18 de May de 2024, de OneWire Library: https://www.pjrc.com/teensy/td_libs_OneWire.html

Urquiza, M. C., Ghiordy, Ramirez, J., Cunha, A., Marano, S., fredolaredo, . . . per1234. (21 de Marzo de 2022). *github.com.* Recuperado el 18 de Mayo de 2024, de MQSensorsLib: https://github.com/miguel5612/MQSensorsLib

7. *Apéndices*

7.1. Tabla de Figuras

Figura 1: Código piloto, con un sensor usando Arduino IDE, y librería Dallas Temp (se muestra temperatura en distintas escalas). 4
Figura 2: Código piloto, con 5 sensores usando Arduino IDE, y librería Dallas Temp. 4
Figura 3: Montaje de micro y sensores 4
Figura 4: Lecturas de humedad DHT11 en Arduino IDE.....5
Figura 5: Montaje de DHT11 con ESP23, misma tabla de pruebas, pero con código aislado para solo medir humedad. 5
Figura 6: Prueba de código piloto extraído del repositorio oficial para lecturas de concentración de CO2. 5
Figura 7: Distribución de signos en la pantalla LCD de 4 filas por 20 columnas. 5
Figura 8: La comunicación con el ESP32 es por medio del protocolo I ^2C, es necesario usar un módulo I2C que permite utilizar dos pines y simplificar la transferencia de información; en

este caso especificando el registro 0x27 y usando Arduino IDE.....6
Figura 9: Con el teclado matricial se mostraron los respectivos valores mostrados en sus celdas, tras presionarlas, se imprimían por serial en el monitor de Arduino IDE. 6
Figura 10: Integración de sensores con el ESP32 y la pantalla. 6
Figura 11: Placa con terminal blocks en toda la periferia para conectar sensores(escala no original). 6
Figura 12: Layout de la placa principal (no a escala original) 7
Figura 13: Placa Terminal block ya soldada 7
Figura 14: Placa principal adaptado para montaje de perifericos 7
Figura 15: Tapa interior del encapsulado 8
Figura 16: Vista superior del dispositivo 8
Figura 17: Vista isométrica (Conexiones no incluidas) 8
Figura 18: Vista isométrica (sin sensores y tapa empotrada) 8
Figura 19: Dispositivo funcionando, armado y conectado 9
Figura 20: Boceto de la interfaz de monitoreo.10
Figura 21: Jerarquía de elementos en HTML ..10
Figura 22: Prototipo de monitor de variables ambientales con html, css y js. 11
Figura 23: Flujo para enviar y actualizar la interfaz de monitoreo con AJAX 11
Figura 24: Prueba de hardware y monitor utilizando valores de prueba 12
Figura 25: Monitor mostrando los valores de prueba.....12
Figura 26: Algoritmo incluyendo conexión a múltiples redes, cambio de IP por nombre constante y uso de SPIFFS.....12
Figura 27: estructura del objeto JSON RTDB. Timestamp se refiere a la marca de tiempo, por cada marca de tiempo se tienen anidados 13
Figura 28: Flujo de información desde sensores hasta Interfaz 14
Figura 29: Solicitudes ida y vuelta incorporadas con JavaScript y Firebase para la interacción de la página web 14
Figura 30: Aspecto final de la aplicación Web.15
Figura 31: Datos RTDB 15
Figura 32: Timestamps ocultos para mayor visibilidad.....15
Figura 33: Archivo JSON descargado de RTDB web app 15
Figura 34: Archivo CSV descargado de RTDB web app 16
Figura 35: Dispositivo recién energizado 16

7.2. Material

7.2.1. *Hardware*

Tabla 1: Tabla de material para Hardware

Nombre	Cantidad
ESP32	1
Sensores DS18B20	4
Sensores DH11	3
Sensores QM135	2
Pantalla LCD	1
Teclado matricial 4 filas, 20 columnas	1
Modulo I2C	1
Regulador de voltaje con disipador de calor 7805CT	1
Capacitores 0.1uF	2
Resistor 5 Ω 1W	1
Resistor 1 kΩ 1/2W	2
Resistor 4.7 kΩ	1
Resistor 3.3 kΩ	2
Batería 9V 600mAh	1
Tiras de header hembra 40 pines	2
Tira de header macho 40 pines	1
Caja plástica (contenedor genérica)	1
Placa fenólica perforada 15x9 cm	1
Cables y jumpers	1
Soldadura Cautín	1

Pasta para soldar	1
Placa perforada 3x7 cm	1
Multímetro	1

7.2.2. *Software*

Tabla 2: Tabla Softwares usados

Nombre
Github
Firebase RTDB y Hosting
VS Code
Extensión PlatformIO (Arduino framework)
NodeJS (vincular con Firebase)
Fusion 360
ChatGPT

7.2.3. *Librerías*

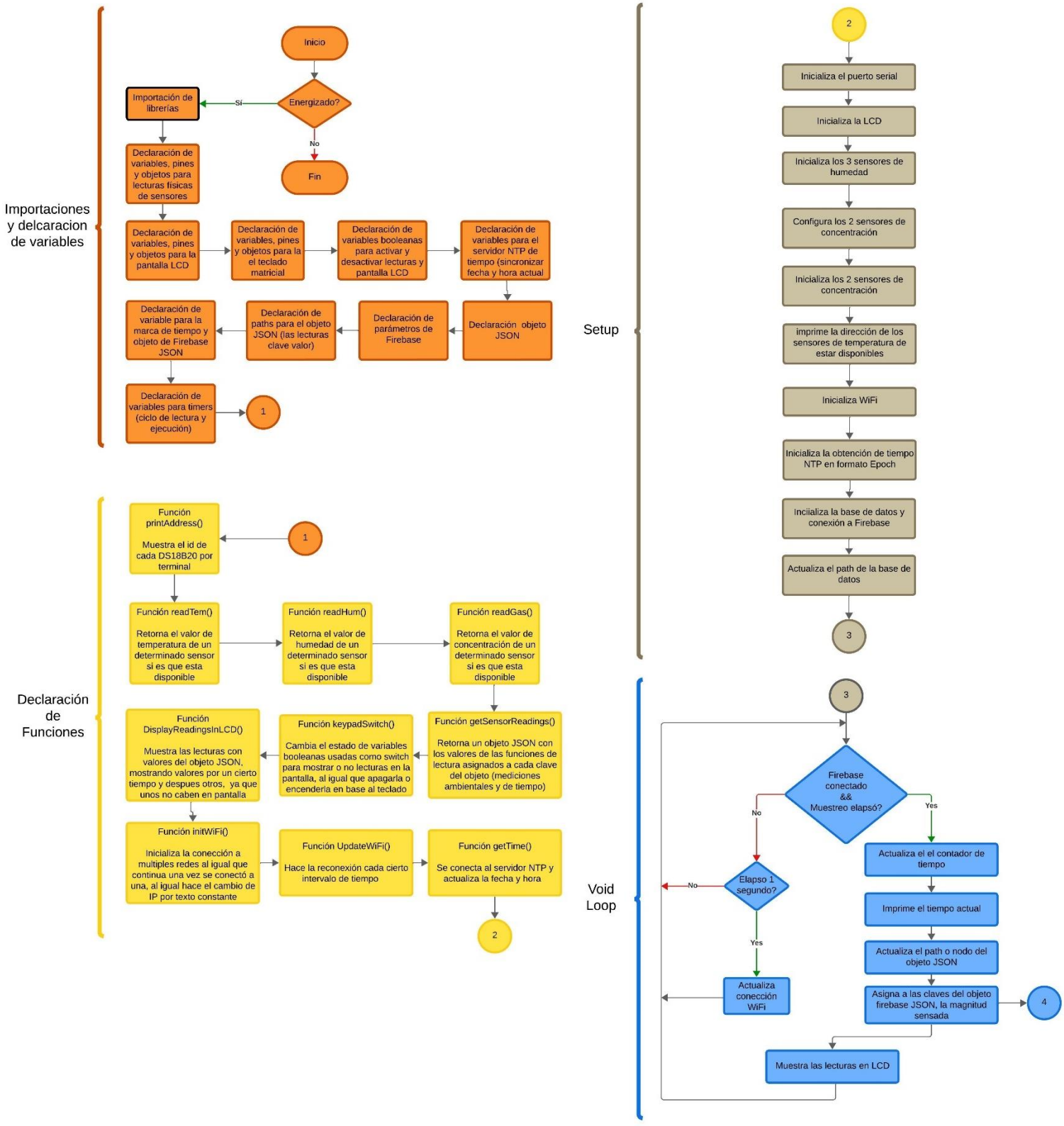
Tabla 3: Tabla de librerías usadas a lo largo del desarrollo.

Nombre	Descripción
LiquidCrystal I2C by Frank de Brabander	LCD
OneWire by Jim Studt	bus wire for DS18B20
DallasTemperature by Miles Burton	DS18B20
DHT sensor library by Adafruit	DHT11
Adafruit Unified Sensor	DHT11
Keypad by Mark Stanley	4X20 keypad
MQUnifiedsensor	MQ135 gas sensor
ESPAsyncWebServer	Servidor asíncrono (no usado actualmente)
AsyncTCP	Servidor asíncrono (no usado actualmente)
WifiMulti	Conexión a múltiples redes
Arduino_JSON by Arduino	Estructura tipo JSON para organizar las lecturas y mostrarlas en la LCD, originalmente usado con AJAX para mostrar datos en la página con host local
NTPClient by Fabrice Weinberg	Obtener fecha y hora de servidor de tiempo
time	Built in, formato de tiempo y hora
ESPmDNS	Asignar un nombre constante a la dirección IP del servidor asíncrono (no usado actualmente)
SPIFFS	Acceso a contenido de documentos cargados al MCU diferente del código principal (no usado actualmente)
Firebase Arduino Client Library for ESP8266 and ESP32	Conexión y envío de datos a la nube

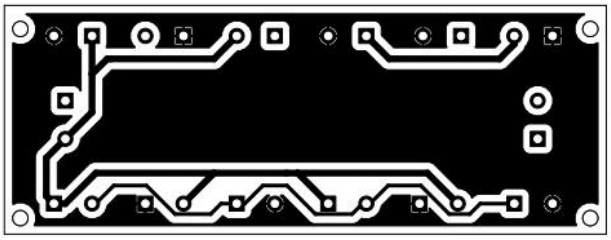
7.3. Repositorio

https://github.com/Jultromix/ESP32-4FUN.git

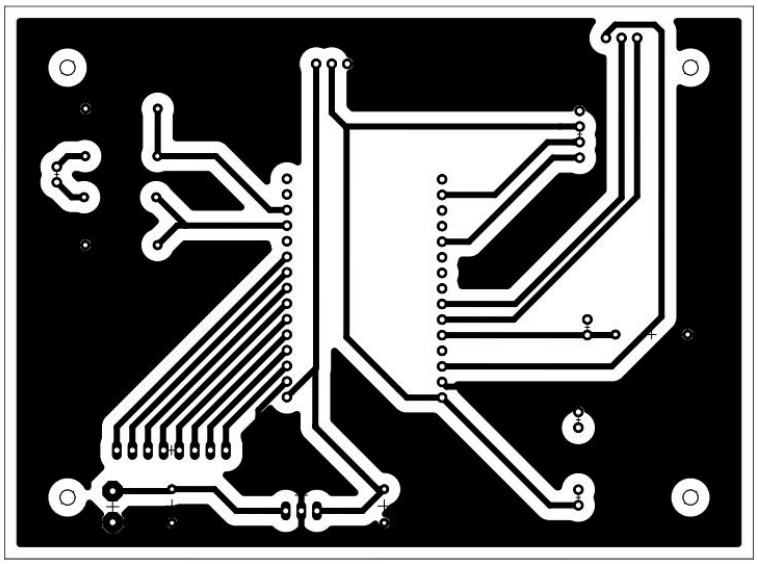
7.3.1. Diagrama de Flujo – código ESP32



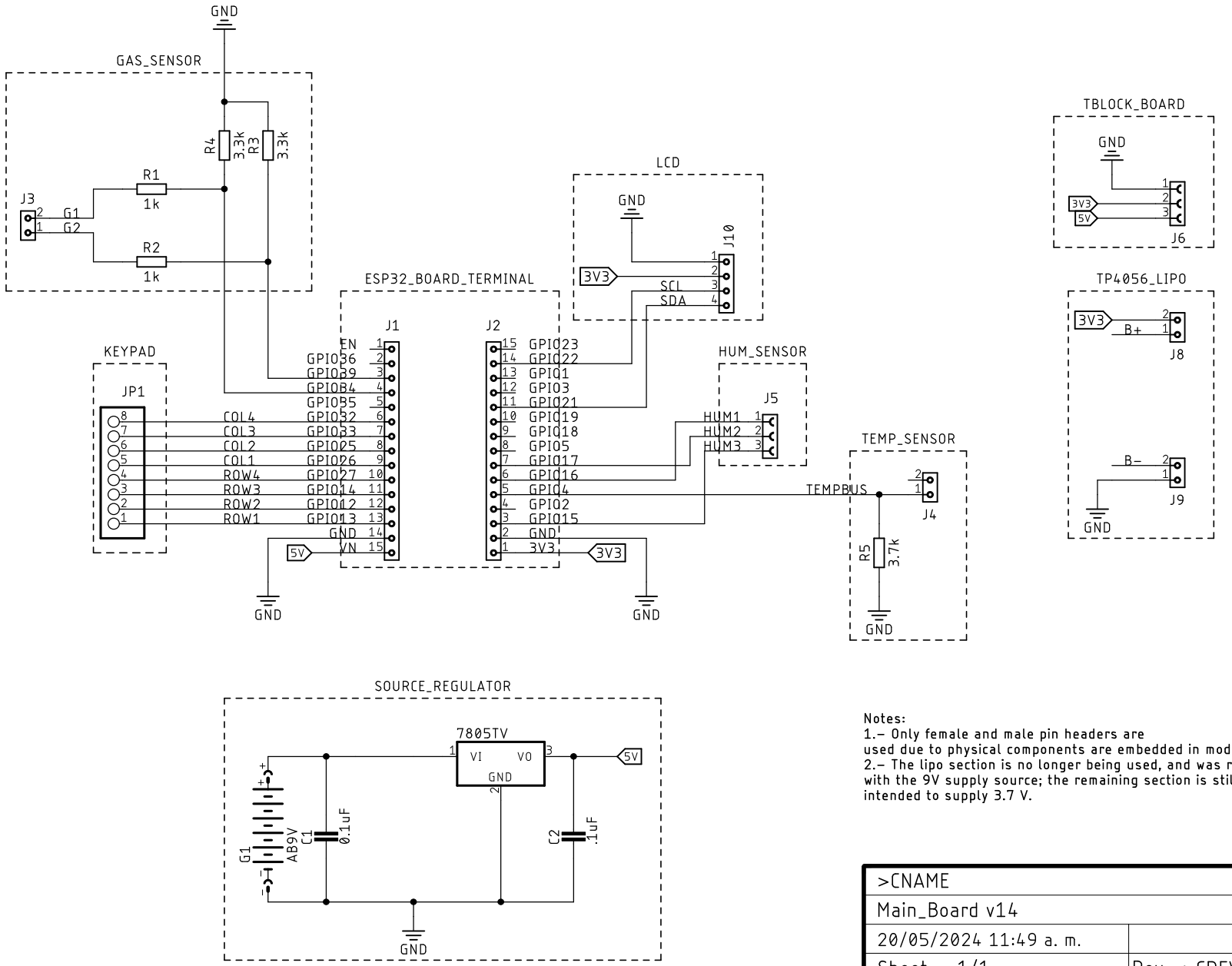
7.3.2. PCB Layout Terminal block



7.3.3. PCB Layout Circuito Principal

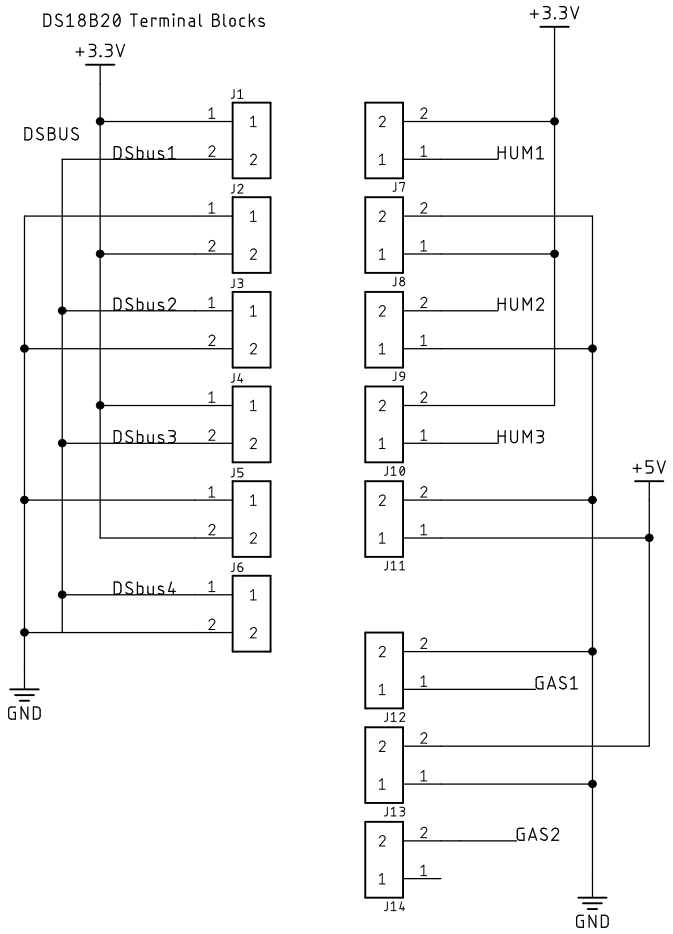


7.3.4. Diagramas de conexiones y Dibujos de ensamble

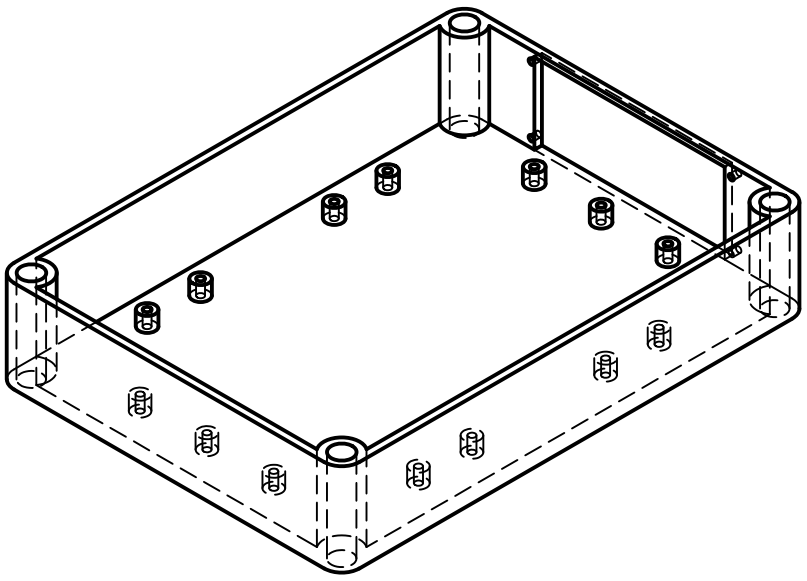


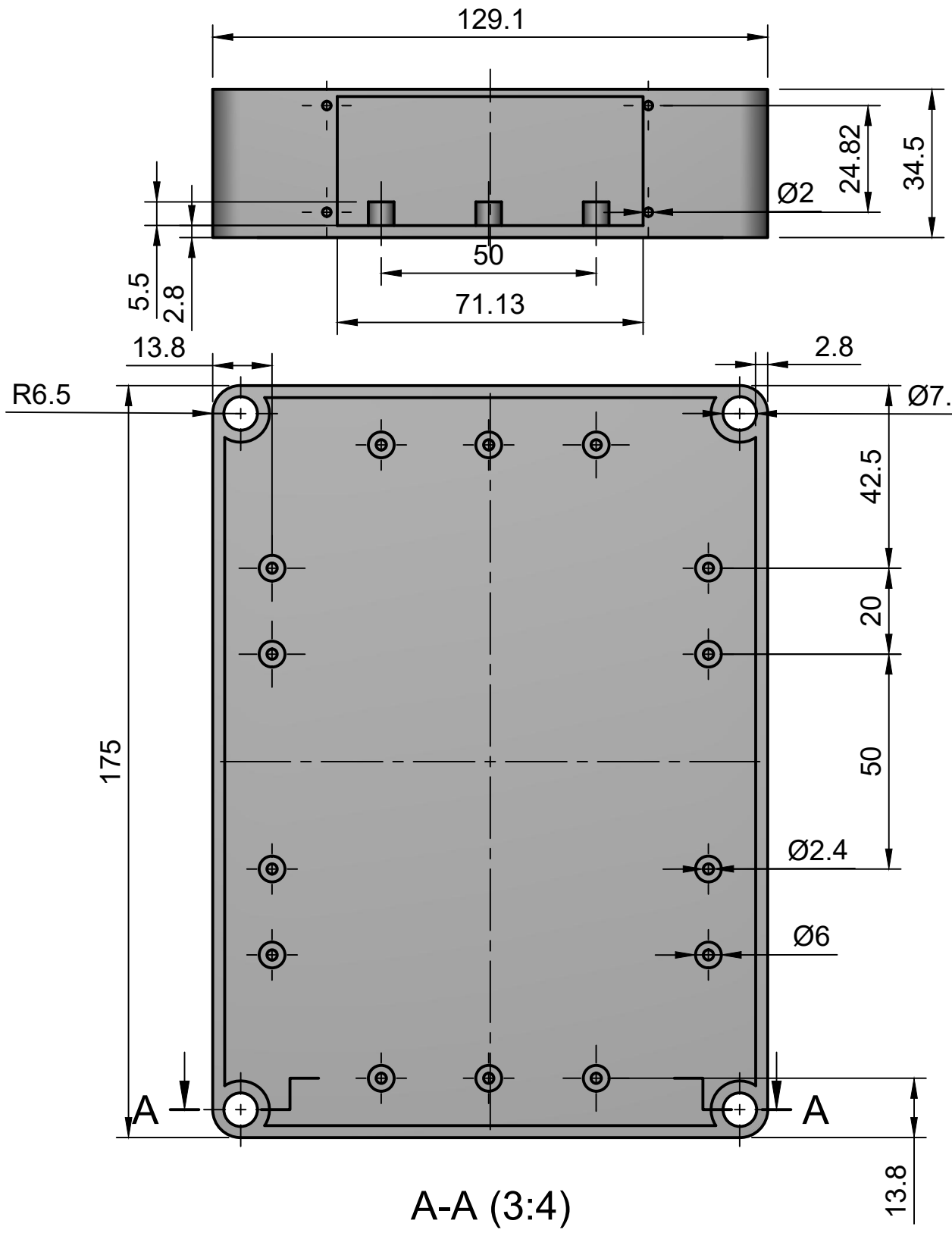
Notes:
1.– Only female and male pin headers are used due to physical components are embedded in modular units.
2.– The lipo section is no longer being used, and was replaced with the 9V supply source; the remaining section is still available and intended to supply 3.7 V.

>CNAME	
Main_Board v14	
20/05/2024 11:49 a. m.	
Sheet: 1/1	Rev: >CREVISION

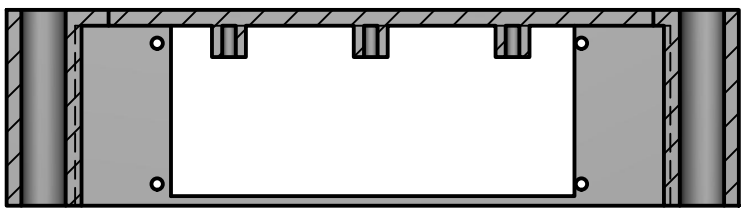


>CNAME TBLOCK BOARD	
Esquema v9	
20/05/2024 11:36 a. m.	
Sheet: 1/1	Rev: >CREVISION

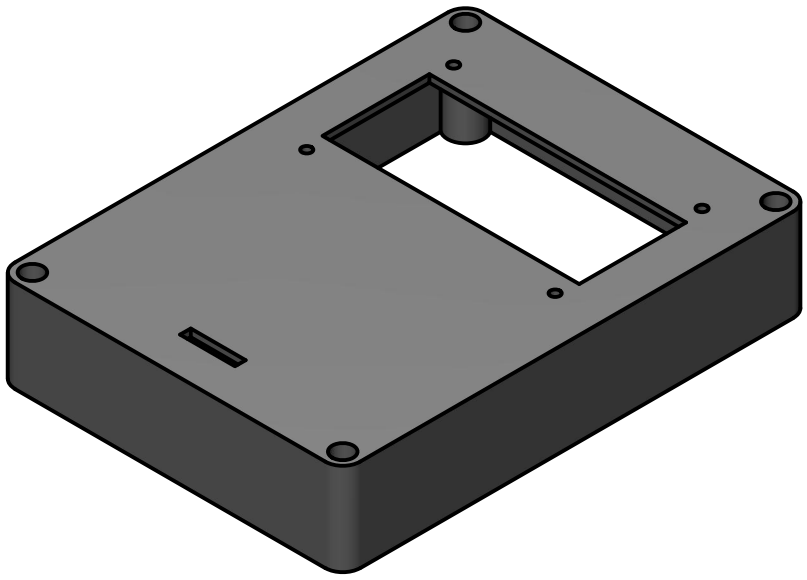




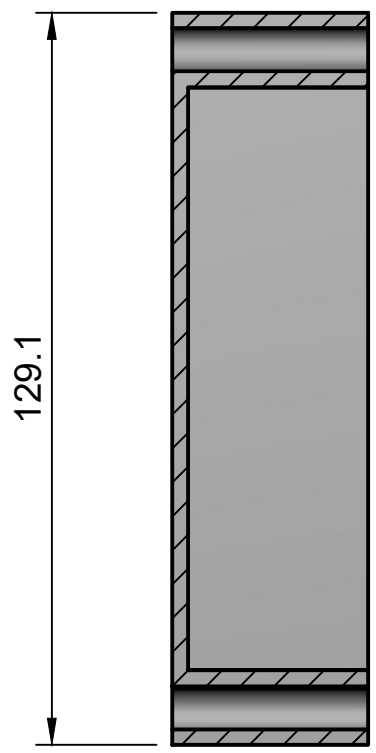
A-A (3:4)

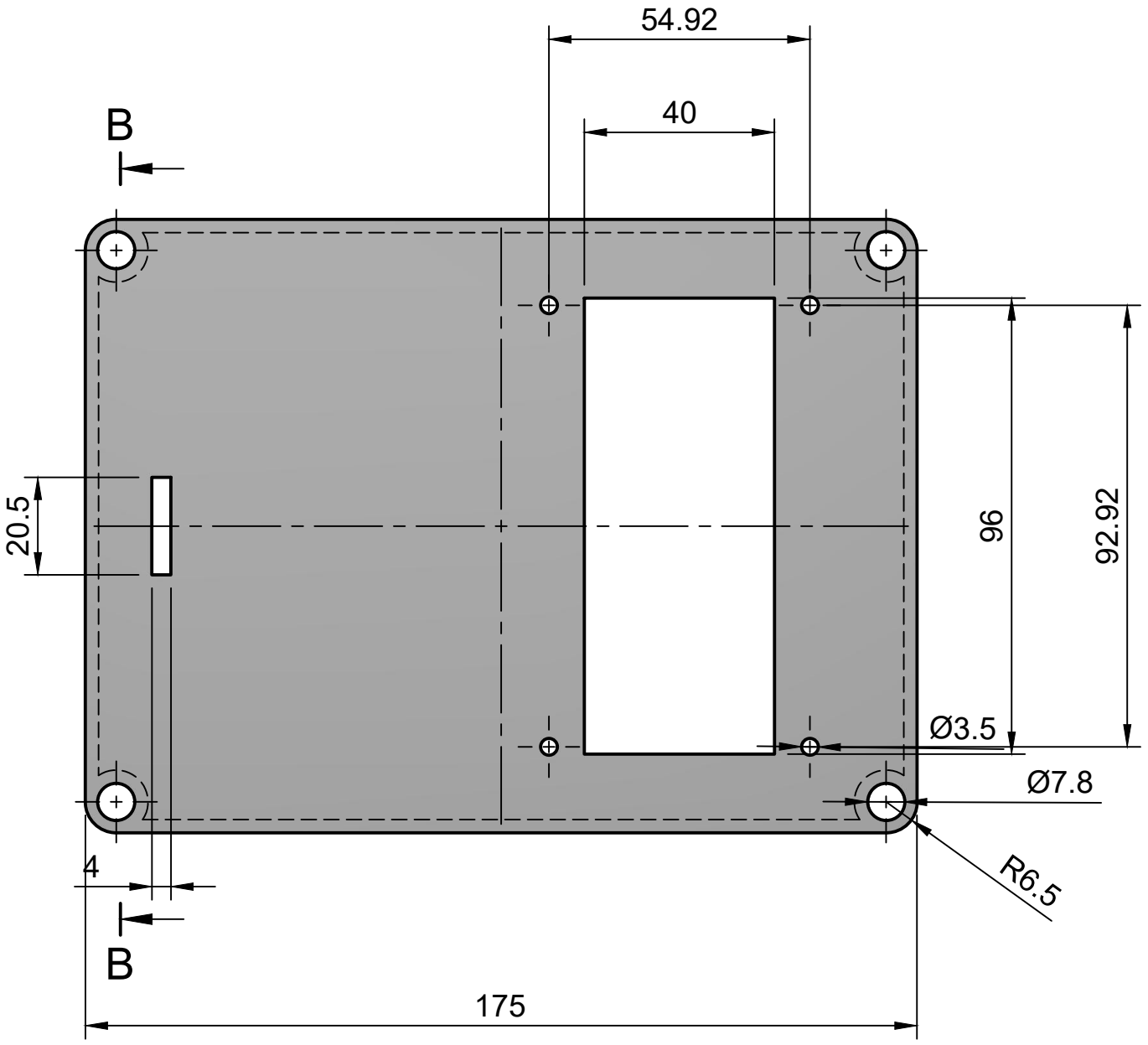


Dept.	Technical reference	Created by Julio Moreno 27/04/2024	Approved by
		Document type	Document status
		Title: Container_Bottom	DWG No.
			Rev. / Date of issue / Sheet 1/4

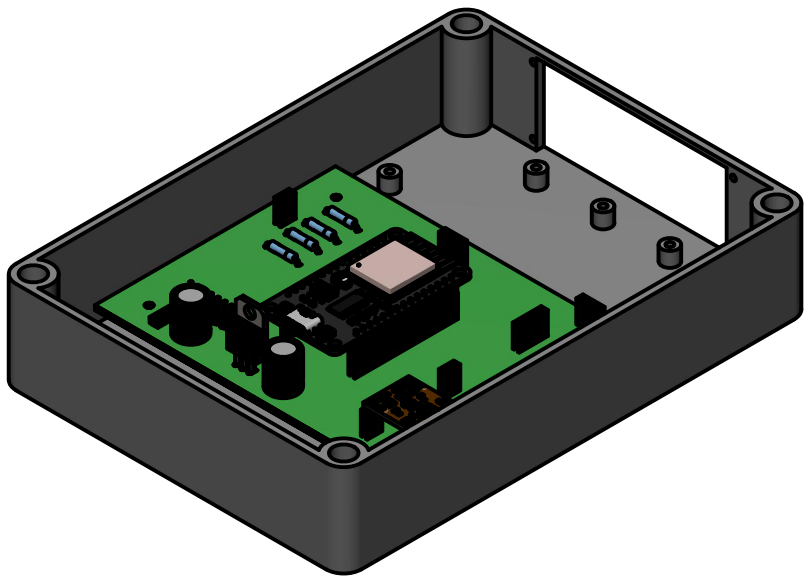


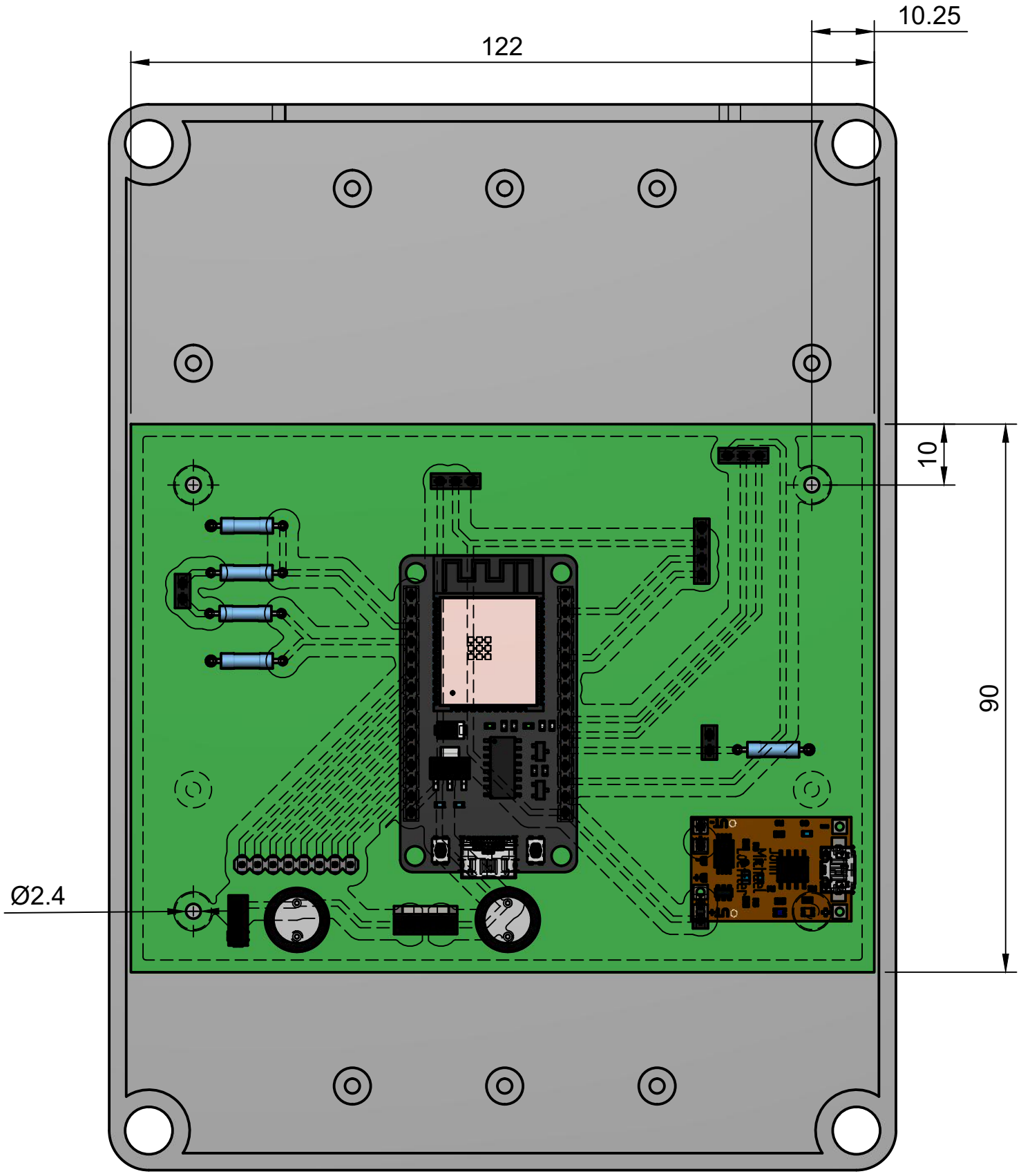
B-B (3:4)





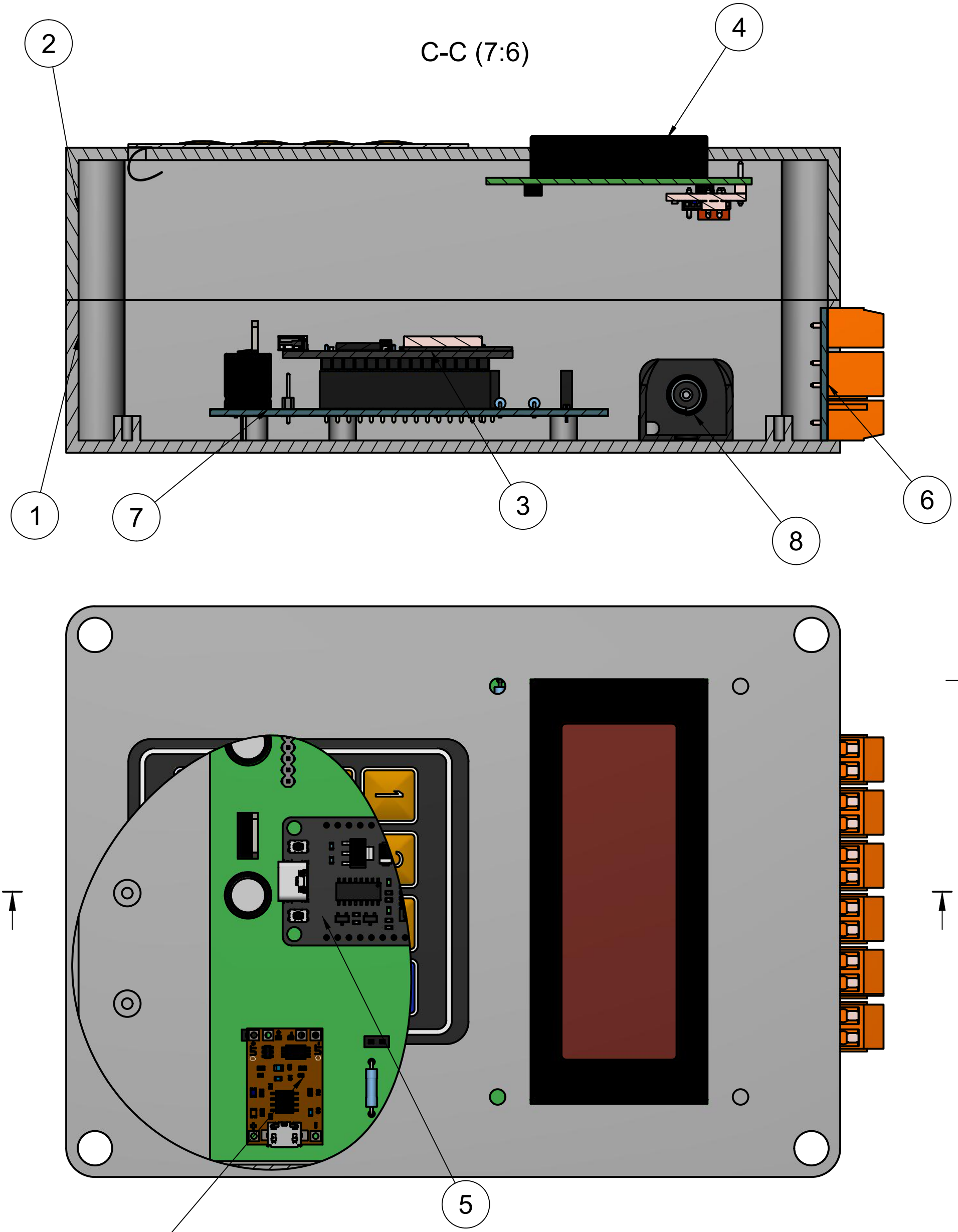
Dept.	Technical reference	Created by Julio Moreno 27/04/2024	Approved by
		Document type	Document status
		Title: Container_Bottom	DWG No.
			Rev. / Date of issue / Sheet 2/4





Dept.	Technical reference	Created by	Approved by
		Julio Moreno 27/04/2024	
		Document type	Document status
		Title: Container_Bottom	DWG No.
			Rev. / Date of issue / Sheet: 3/4

C-C (7:6)



C

C

Dept.	Technical reference	Created by Julio Moreno 27/04/2024	Approved by
		Document type	Document status
		Title Container_Bottom	DWG No.
			Rev. / Date of issue / Sheet 4/4